Sri Lanka Institute of Information Technology

3rd Year - 2nd Semester 2026 | IT3060

Human Computer Interaction

HISTORIA

Historical Walking-Tour Route Planner Mobile Application

MILESTONE 03

Mobile App Implementation & Final Evaluation

Group Number: Y3S2_WE_82 | Group Name: WE_82

Student ID	Student name	Assigned workload / main features
IT23678668	DE SILVA D V K D	Historical discovery, community feed and post creation, multi-stop tour planning and progress.
IT23687882	Maduvinda D K A	User management, tourist/guide profiles, OTP/recovery, smart travel buddy matching and chat.
IT23687646	Lekamge U L C D	Guide-booking payment options and checkout; payment status; ratings and reviews.
IT23667778	Siriwardhana H L	Local-guide discovery/booking, scheduling, meeting-point logistics and guide tracking.

Table of Contents

1. Executive Summary 3

2. Milestone 01 Summary 4

3. Milestone 02 Summary 6

4. Technology Stack Selection and Justification 7

5. System and Application Architecture 8

6. Implementation Details and Working Application 10

7. Prototype Fidelity and Deviations 23

8. Traceability Matrix 24

9. Functional Testing 25

10. Usability Testing 35

11. Issues, Fixes and Recommendations 36

12. Overall Time Schedule (Gantt Chart) 37

13. Conclusion and Lessons Learned 38

14. References 38

Appendix - Repository, Build and Run Evidence 39

1. Executive Summary

HISTORIA is a Sri Lanka-focused historical walking-tour mobile application designed to bring heritage-site information, personal tour planning, local tour guides and social travel coordination into a single experience. The original project was introduced in Milestone 01 as the “Historical Walking-Tour Route Planner Mobile Application” by Group Y3S2_WE_82. The app name HISTORIA and its connected prototype flows were developed during Milestone 02.

Milestone 01 studied fragmented visitor information, route-finding challenges, limited historical explanations, difficulties locating nearby facilities and unreliable connectivity. The team collected 37 usable questionnaire responses and documented Galle Fort observations, stakeholder groups, personas, empathy maps, user stories, a journey map and evidence-led requirements. Milestone 02 translated these needs into sketches, variant-selection rationales, wireframes, low-fidelity prototypes and a connected high-fidelity design.

The development architecture uses a Flutter/Dart client communicating with a Spring Boot/Java REST API and PostgreSQL storage. Spring Security, JWT authentication and role-aware endpoints are used for access control. Three application roles are defined: TOURIST, GUIDE and ADMIN. Development work discussed by the team covers account registration/verification/recovery, profiles, historical-place discovery, multi-stop tours, community posts, travel-buddy matching/chat, guide applications and booking, payment-review workflows and status feedback. Every module must still be checked in the final group-integrated build before its status is recorded as “Passed”.

This final-report draft presents implementation interfaces and candidate API evidence, a prototype-to-implementation comparison, a requirement traceability matrix, functional test scripts, a five-participant minimum usability-test framework, defect tracking, a time schedule and an evidence checklist. The most important final verification is the full guide-booking-to-payment path: local development may use a demo checkout, which is not equivalent to actual card processing.

TO COMPLETE / VERIFY: Enter final total tests executed/passed, real device evidence, five or more usability participants, outstanding defects and commit hash after running final verification.

2. Milestone 01 Summary

2.1 Problem Context

Visitors to historical attractions often need to switch between general map services, online articles, physical signs and local guidance. This fragmented experience makes it difficult to understand what a visitor is seeing, choose a suitable route and locate facilities in a single journey. The Group 82 research highlighted inconsistent connectivity at heritage sites and demand for reliable navigation and engaging historical stories. The chosen solution was a mobile app combining historical information, route planning, nearby facilities and access to guides. [SOURCE: M1]

2.2 Stakeholders

Group	Stakeholders	User needs
Primary	Local and international tourists, walking-tour visitors and students.	Discover places; understand heritage; plan and follow manageable routes.
Secondary	Professional tour guides, local shops/service providers, heritage personnel.	Trusted guide visibility, bookings, accurate locations and visitor support.
Tertiary	Administrators and tourism/heritage organisations.	Operational reliability, governance, content quality and responsible tourism.

2.3 Research Methodology and Participant Profile

Milestone 01 used an online questionnaire with quantitative and free-text questions, observation around Galle Fort, interviews and thematic analysis. The final questionnaire sample contained 37 usable responses. Thirty respondents (81.1%) were aged 18-24, 29 (78.4%) were students, 24 (64.9%) had previously joined a walking tour and 28 (75.7%) were interested/very interested in historical places. Responses were anonymised P01-P37. The sample is exploratory and youth/student-weighted, rather than representative of all tourists.

Research finding	Count	Percentage
Lack of sufficient historical details	22 / 37	59.5%
Nearby facilities difficult to locate	20 / 37	54.1%
Problems planning/following walking routes	18 / 37	48.6%
Weak/unreliable internet	21 / 37	56.8%
GPS navigation requested	33 / 37	89.2%
Offline maps rated useful/very useful	35 / 37	94.6%
Normally visit with family	26 / 37	70.3%

These results justify combining rich place descriptions, clear wayfinding, tour planning and reliable on-site information. Offline mapping and multilingual support were strongly suggested research needs, but their presence in the final integrated app must be verified rather than assumed. [SOURCE: M1]

2.4 User-centred Findings, Personas and HMW

The research identified five repeated qualitative themes: fragmented information; unstable internet; interest in memorable historical stories; difficult route planning at complex sites; and the potential for guide services to complement rather than compete with a digital guide. The three research personas were Nethmi Perera, Dilan Fernando and Kavindi Silva. These represented needs for approachable discovery, practical group/family planning and relevant heritage learning. [SOURCE: M1]

HMW ID	Evidence-driven design opportunity
HMW 01	Keep route guidance useful when mobile connectivity weakens; explore offline/saved maps and recoverable guidance.
HMW 02	Tell historical stories in a meaningful way when a physical guide is not present.
HMW 03	Combine walking routes, nearby attractions and practical facilities in a single planning journey.
HMW 04	Make the experience understandable across languages and local Sri Lankan contexts.

2.5 Main Requirements Carried into Milestone 02

The Milestone 02 traceability sheet lists FR01-FR15; some entries are explicitly marked “provisional mapping” rather than already-approved Milestone 01 requirement IDs. The report preserves this distinction. Functional themes include authentication, user/guide accounts, place information, community posts, multi-destination tours, progress, guide approval/profiles, notifications, buddy matching and communication, guide selection, booking and guide meeting logistics. Payment and review interfaces are fully described as team member workload in Milestone 02, but formal FR IDs should be checked against the original requirement register. [SOURCE: M2]

Non-functional quality targets include clear navigation, responsive layout, readable text and touch controls, truthful status feedback, secure user and payment data, reliable validation, acceptable API performance, device compatibility and resilient behaviour when networks are weak. Numerical performance thresholds and security sign-off are not recorded in these milestone source documents; define and verify them before final submission.

3. Milestone 02 Summary

3.1 From Sketches to High-Fidelity Prototype

The Group 82 design followed a human-centred sequence: requirement/interface mapping → multiple hand-drawn sketch variants → selection rationale → wireframes and connected low-fidelity flows → high-fidelity prototype. The selected designs emphasised direct user actions, uncluttered booking summaries, readable guide details, focused registration, visible system status and consistent navigation.

Workstream / member	Documented selected design direction
DE SILVA D V K D	Discovery feed; direct “Create Post” interface; search-and-add tour planning; simplified progress and summary.
Maduvinda D K A	Focused login/OTP/recovery, readable tourist/guide profiles and travel buddy match display.
Lekamge U L C D	Summary-first checkout, clear payment success/failure states and straightforward star ratings/reviews.
Siriwardhana H L	Detailed guide list cards, selectable packages, landmark-based meeting instructions and arrival countdown.

3.2 Prototype Navigation and Workload Coverage

The approved prototype includes flows for sign-in, verification, account recovery, profile management, historical-site discovery, post creation, tour planning and progress, buddy recommendations/chat, local-guide choice, booking customisation, meeting-point information, checkout, payment result and ratings/reviews. Travel buddy match percentage and shared interests were intended to be understandable, and guide cards were expected to show language, availability and service details. [SOURCE: M2]

Milestone 02 records a high-fidelity prototype video and full prototype folder, included in Section 14. These are design references, not evidence that the matching Flutter implementation works. The next milestone must match every critical UI flow to a real screen, backend response and executed test.

4. Technology Stack Selection and Justification

Technology / tool	Role in HISTORIA	Justification / verification
Flutter and Dart	Mobile frontend, widgets, navigation, responsive forms.	Reusable cross-screen UI and Android emulator/physical-device testing.
Spring Boot / Java	REST controllers, services, validation and transactions.	Separation of business logic from UI; integrated JSON APIs.
PostgreSQL	Users, guide profiles, places, tours, bookings, payments and posts.	Relational consistency, constraints and reliable persistence.
Spring Security / JWT	Authentication, role-based route restrictions.	Protected actions for TOURIST, GUIDE and ADMIN; verify token handling.
BCrypt password hashing	Secure password storage.	Passwords should not be stored or logged in plaintext.
Figma, sketches	Prototype and UX evidence.	Traceable design decisions from Milestone 02.
Git / GitHub	Source control, feature branches and integration.	Supports collaboration, pull requests and final reproducible commit.
Postman / API client	Manual API verification.	Demonstrates requests, status codes and DB-backed CRUD.
Android emulator / phone	End-to-end UI validation.	Supports screen-size, keyboard and network troubleshooting.

The frontend and backend are kept separate to allow parallel work by four members. The backend provides API contracts, validates submitted data and controls persistence. Verification must cover invalid data, unauthorised access, failed connectivity and changes made through the interface. The exact final Java, Spring Boot, Flutter and PostgreSQL version numbers should be copied from the submitted repository, not estimated.

5. System and Application Architecture

The HISTORIA mobile application follows a layered mobile application architecture consisting of the presentation, application, backend, authentication, and database layers. The mobile interface is developed using Flutter and Dart, with a single application that supports multiple user roles, including Tourist, Tour Guide, and Administrator. The app communicates with a Spring Boot backend through RESTful APIs. JWT-based authentication using Spring Security manages user authentication and authorization, while PostgreSQL provides structured storage for application data such as users, historical places, community posts, tours, bookings, reviews, and notifications.

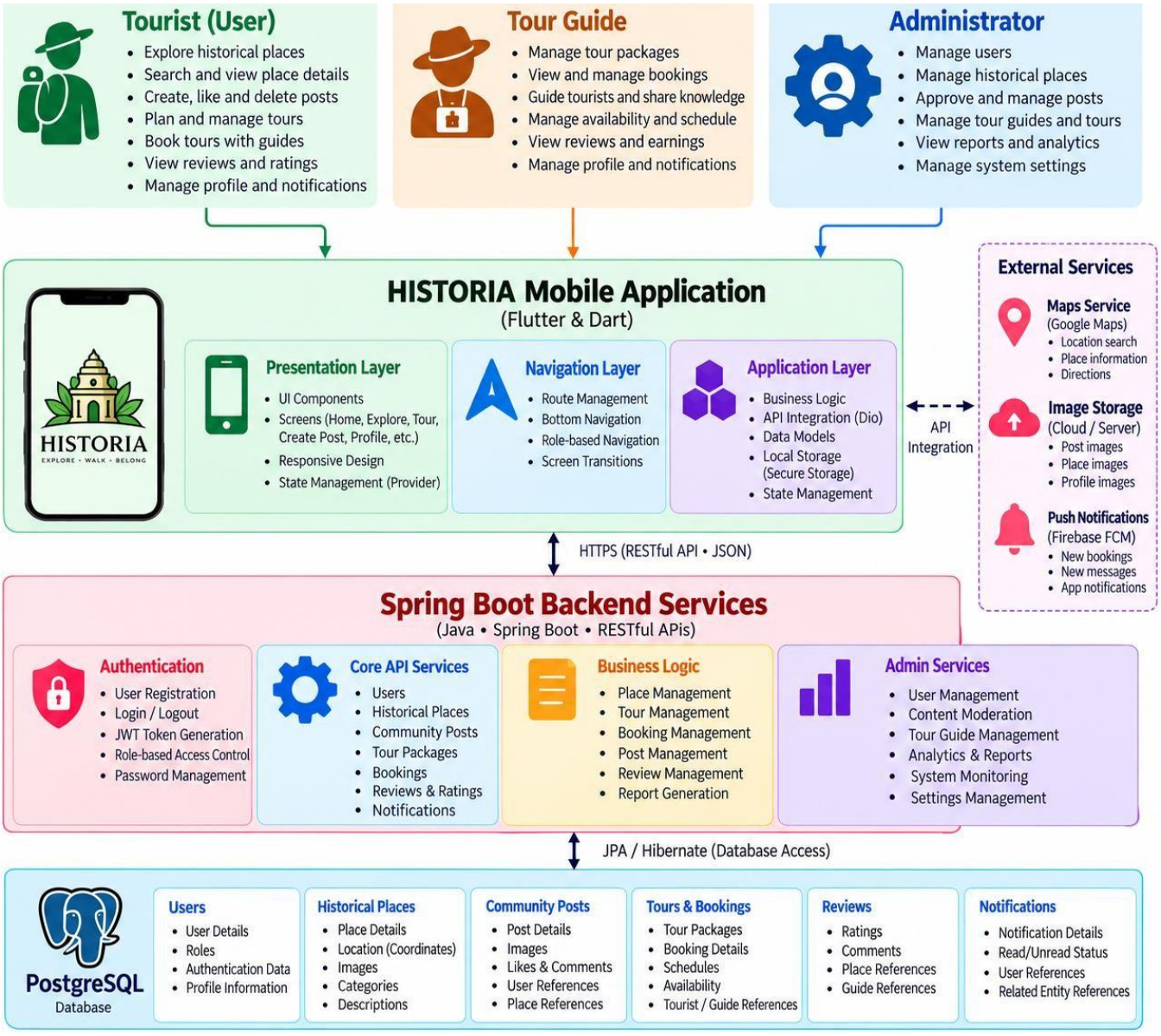


Figure 5.1. HISTORIA system architecture.

5.1 High-Level System Design

CLIENT	API + SECURITY	PERSISTENCE / EXTERNAL
Flutter Android UI Tourist • Guide • Admin	HTTP(S) JSON → Spring Boot controllers Spring Security / JWT filter	PostgreSQL database Users • Profiles • Guides • Tours • Bookings
Login, discovery, tours, buddy chat, guide booking and reviews	Service validation / authorisation Repository / JPA mapping	Notification and OTP email component Development-only payment workflow
Device GPS/location or image selector where applicable	Error handling; DTO request-response mapping	Optional third-party payment/maps providers only when integrated

Typical request path: Flutter screen → API/service class → HTTP request with authenticated token → security filter and controller → service/repository → PostgreSQL → JSON response → Flutter state/UI. Protected operations must derive user identity from the authenticated principal; an ID included in a request body is not evidence of ownership. [DEV NOTE]

5.2 Backend Layers and Representative Data

Layer	Responsibilities	Example artifacts (development discussions)
Security	Login identity, JWT verification, role guards.	JwtAuthFilter, Spring Security, @PreAuthorize.
Controller	Expose API routes and validate transport payloads.	Auth endpoints; historical places; /api/tours; /api/posts; /api/admin/guides.
Service	Apply domain rules; booking/tour and status transitions.	TourService / TourServiceImpl; booking/payment service concepts.
Repository	Fetch and persist domain entities.	UserRepository, PostRepository, BookingRepository.
Entity / DTO	Relational model and JSON-safe payloads.	User, HistoricalPlace, Tour, TourPlace, Post, Booking, Payment.
Exception handling	Consistent errors for invalid data and permissions.	GlobalExceptionHandler / UserException.

5.3 Application Roles and Authorization

Role	Permitted workflow
TOURIST	Self-register; verify account; authenticate; use personal profile, tours, community, buddy matching and guide booking subject to working features.
GUIDE	Access guide-specific profile/application state; maintain guide-specific information and bookings where authorised.
ADMIN	Manage guide applications and guide accounts; access administration screens and protected endpoints.

6. Implementation Details and Working Application

6.1 Member 2-User management, tourist/guide profiles, OTP/recovery, smart travel buddy matching and chat

Member 2: IT23687882 - Maduvinda D K A.

User Authentication and Account Management

The HISTORIA application provides secure user authentication and account management for three roles: Tourist, Guide, and Admin. Tourists can register using their personal details and verify their email addresses through a six-digit OTP. Registered users can log in using their email, username, or phone number. JWT authentication and role-based access control are implemented using Spring Boot Security.

Password Recovery and OTP Verification

The application provides a secure password recovery process using email OTP verification. Users can request password resets using their registered email, username, or phone number. OTP codes expire after five minutes, with resend restrictions and retry limitations to improve security.

Tourist and Guide Profile Management

Tourists and guides can view and update their personal information through dedicated profile interfaces. The application supports profile and cover photographs, personal information updates, and guide application management. User information is managed through Spring Boot REST APIs and stored in PostgreSQL.

Smart Travel Buddy Matching

The Smart Travel Buddy Matching feature helps tourists discover compatible travel companions based on destinations, travel dates, preferred languages, and shared interests. Users can explore potential matches and connect with other tourists planning similar journeys.

Travel Buddy Chat

The integrated messaging feature allows tourists to communicate with their travel buddies. The application provides a conversation inbox and individual chat interfaces where users can exchange messages, discuss travel plans, and coordinate their historical tours.

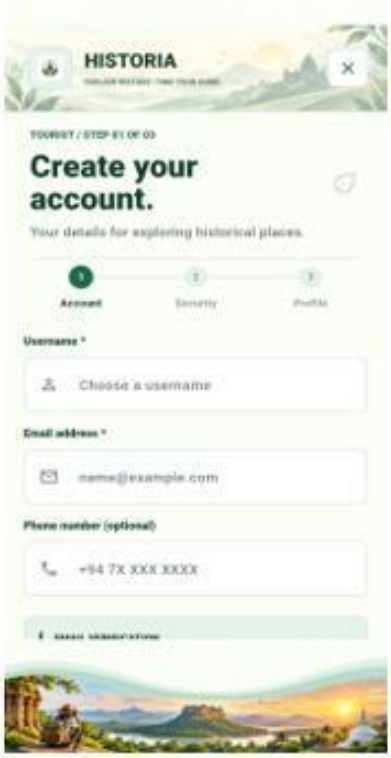
Implementation Technologies

The frontend was developed using Flutter and Dart, while Spring Boot provides REST API services and PostgreSQL manages persistent data. Spring Security and JWT authentication protect user accounts and authorized application operations.

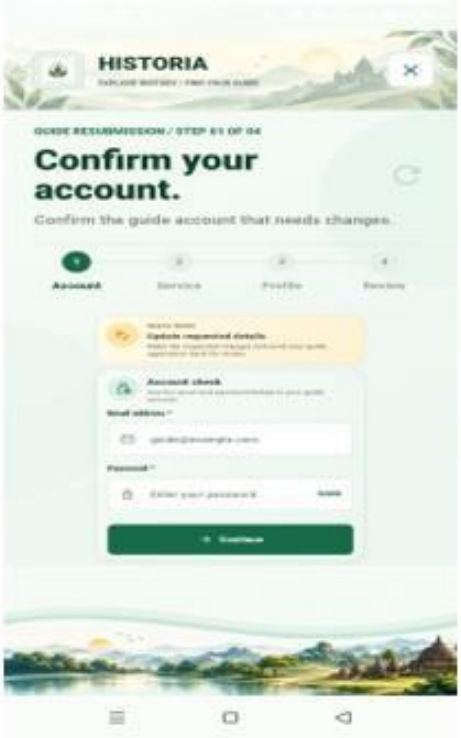
6.1-A Login, sign-up, OTP verification and password recovery

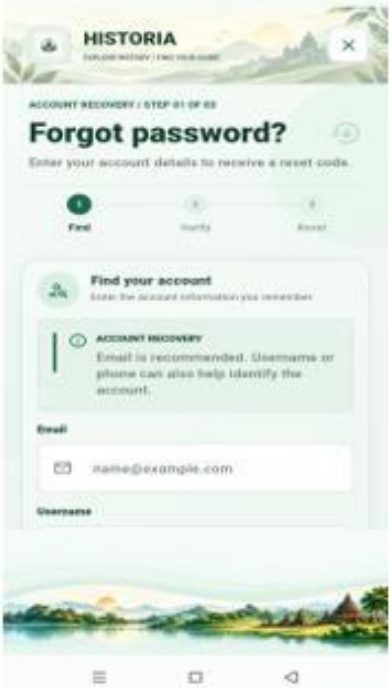




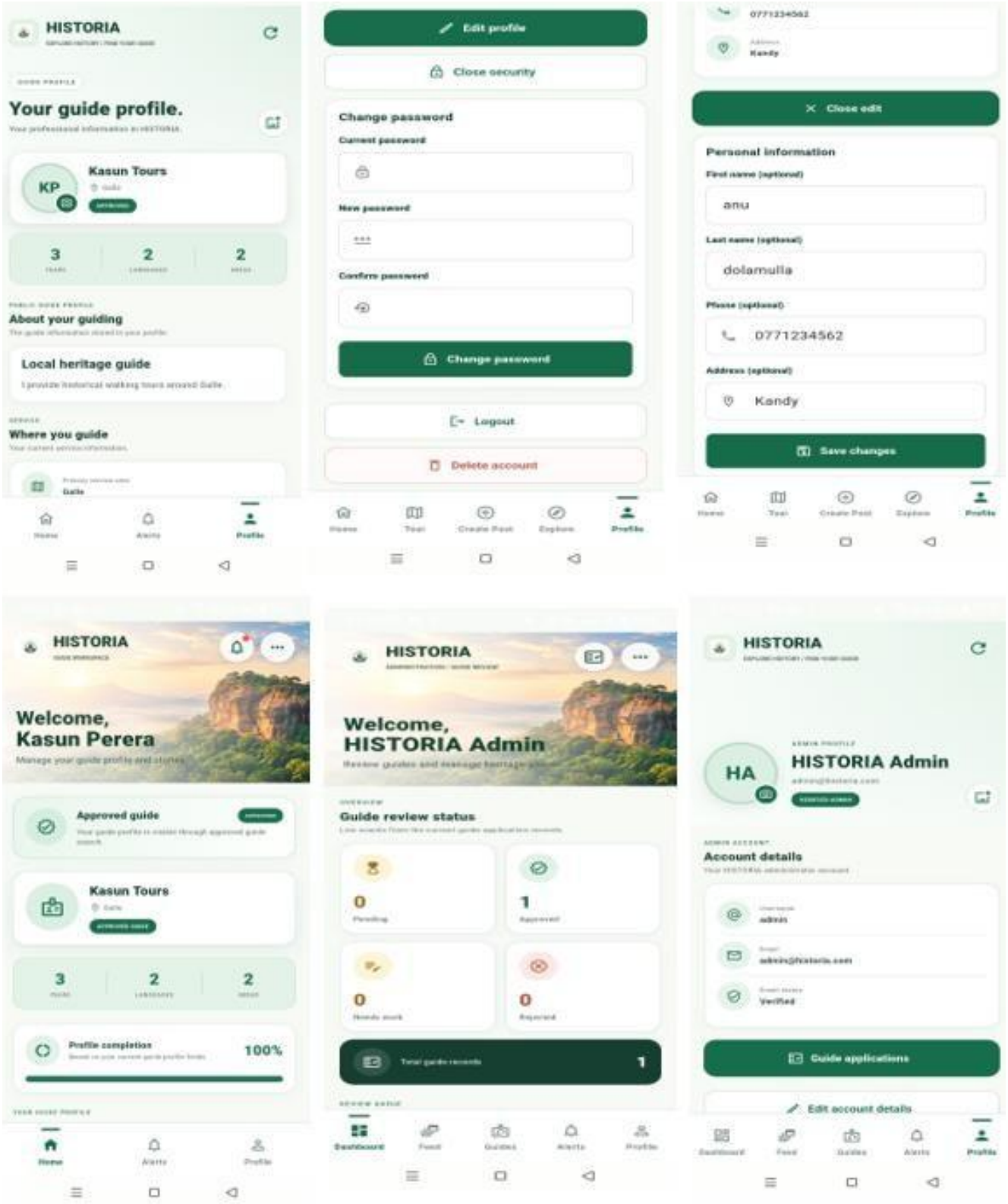






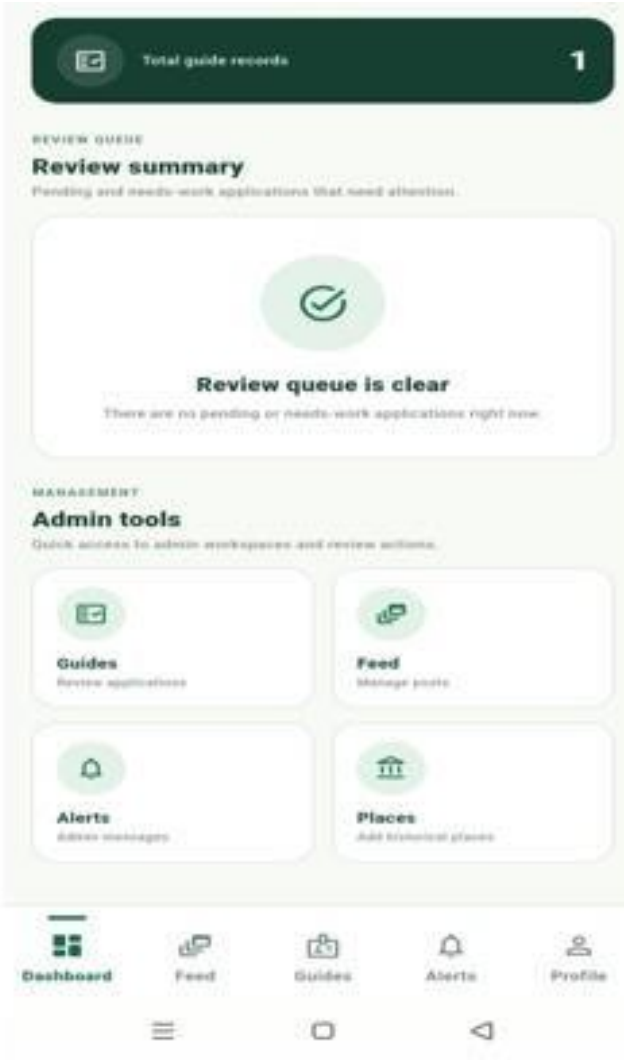


6.1-B Profiles, administration, notifications and home



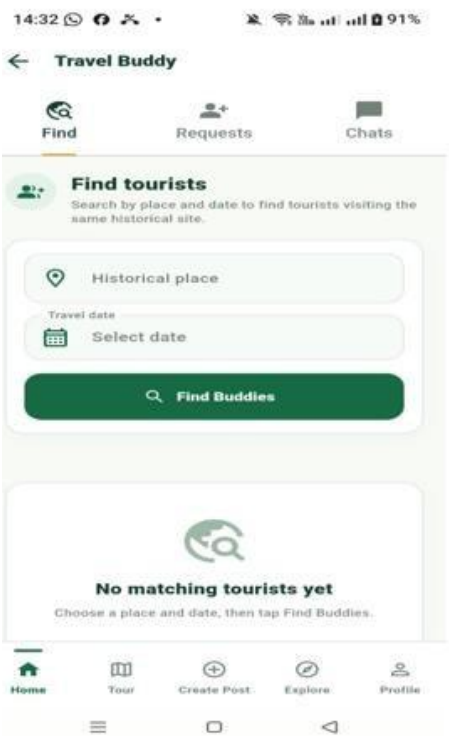
6.1-C Profile and administration screens

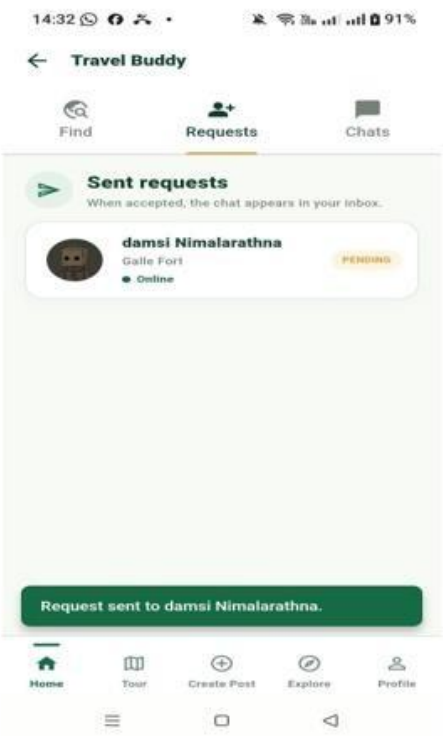


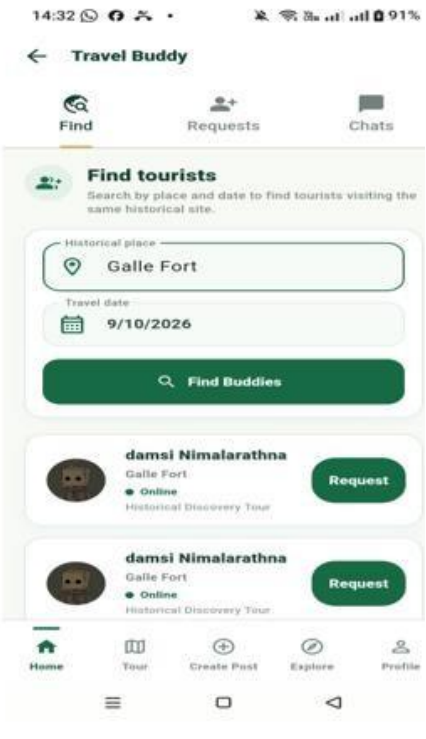


6.2 Member 2 - Smart Travel Buddy Matching and Real-Time Chat

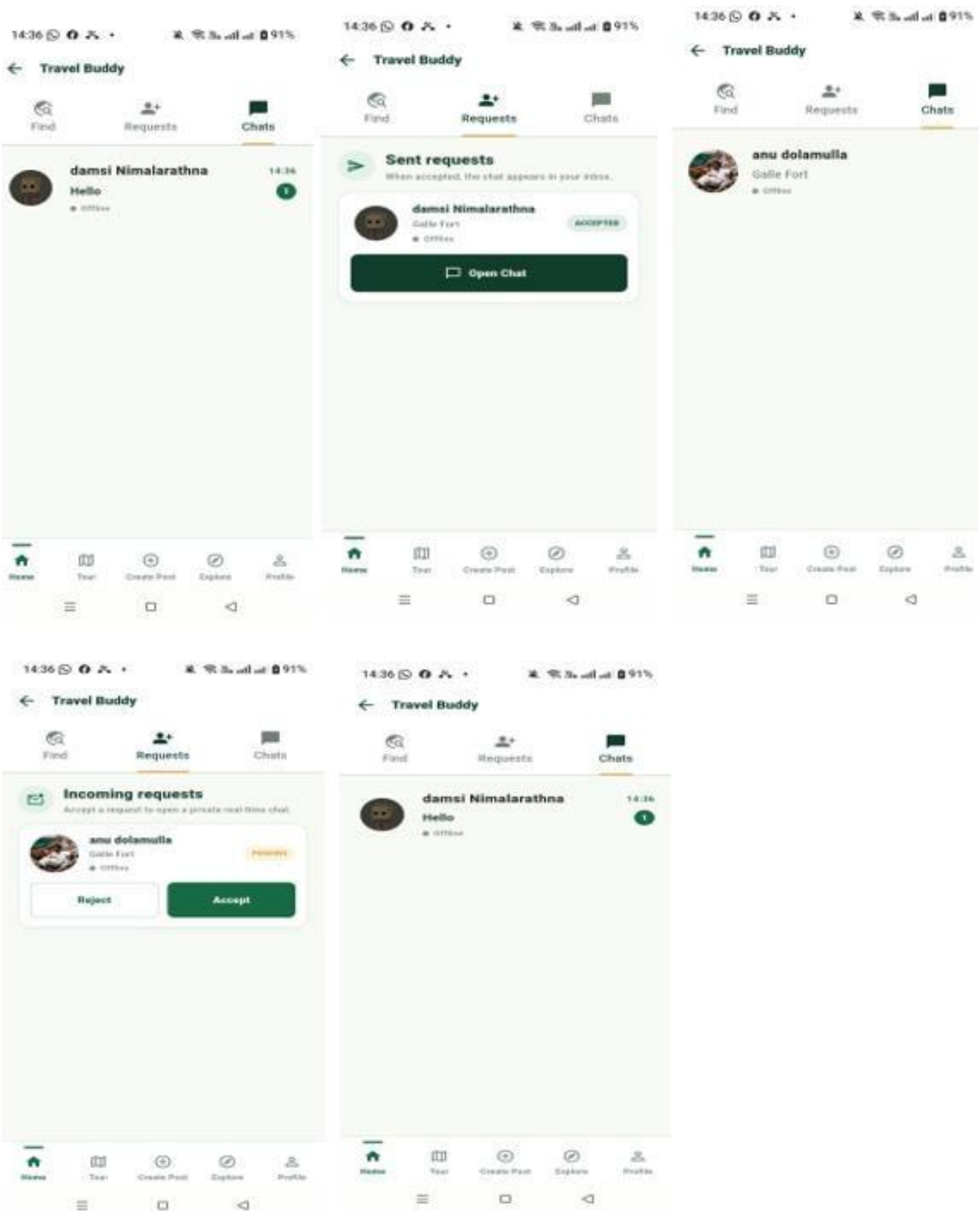
The Milestone 02 prototype includes a matching screen with preference/interest overlap and an acceptance/coordination conversation. During implementation discussions the team added buddy matching, a chat destination and an inbox-like conversation list. Actual real-time delivery, read receipts, chat persistence and refresh/reconnect behaviour must be demonstrated rather than assumed from the existence of a chat page.



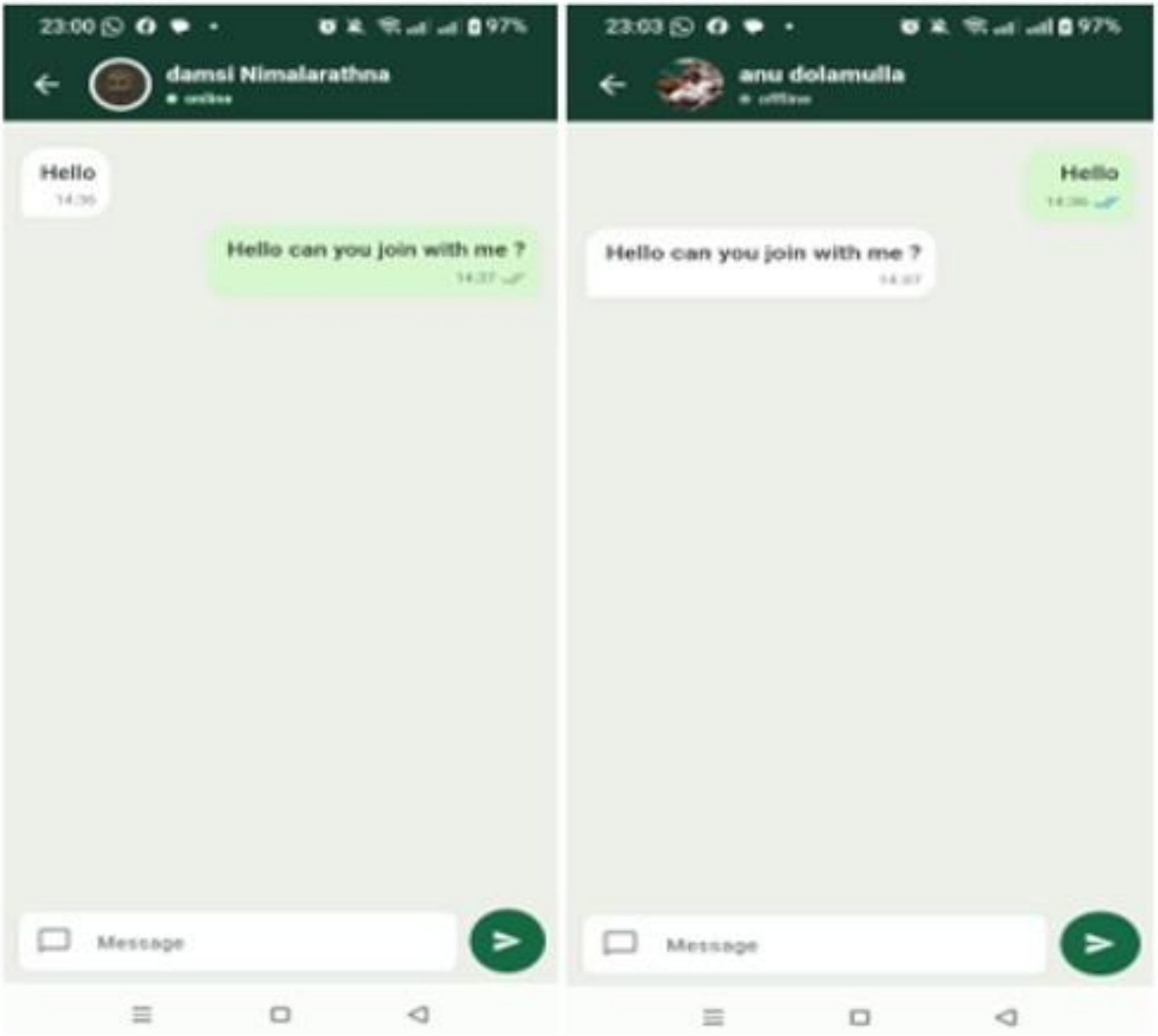




6.2-A Buddy recommendations



6.2-B Chat inbox and two-account message evidence

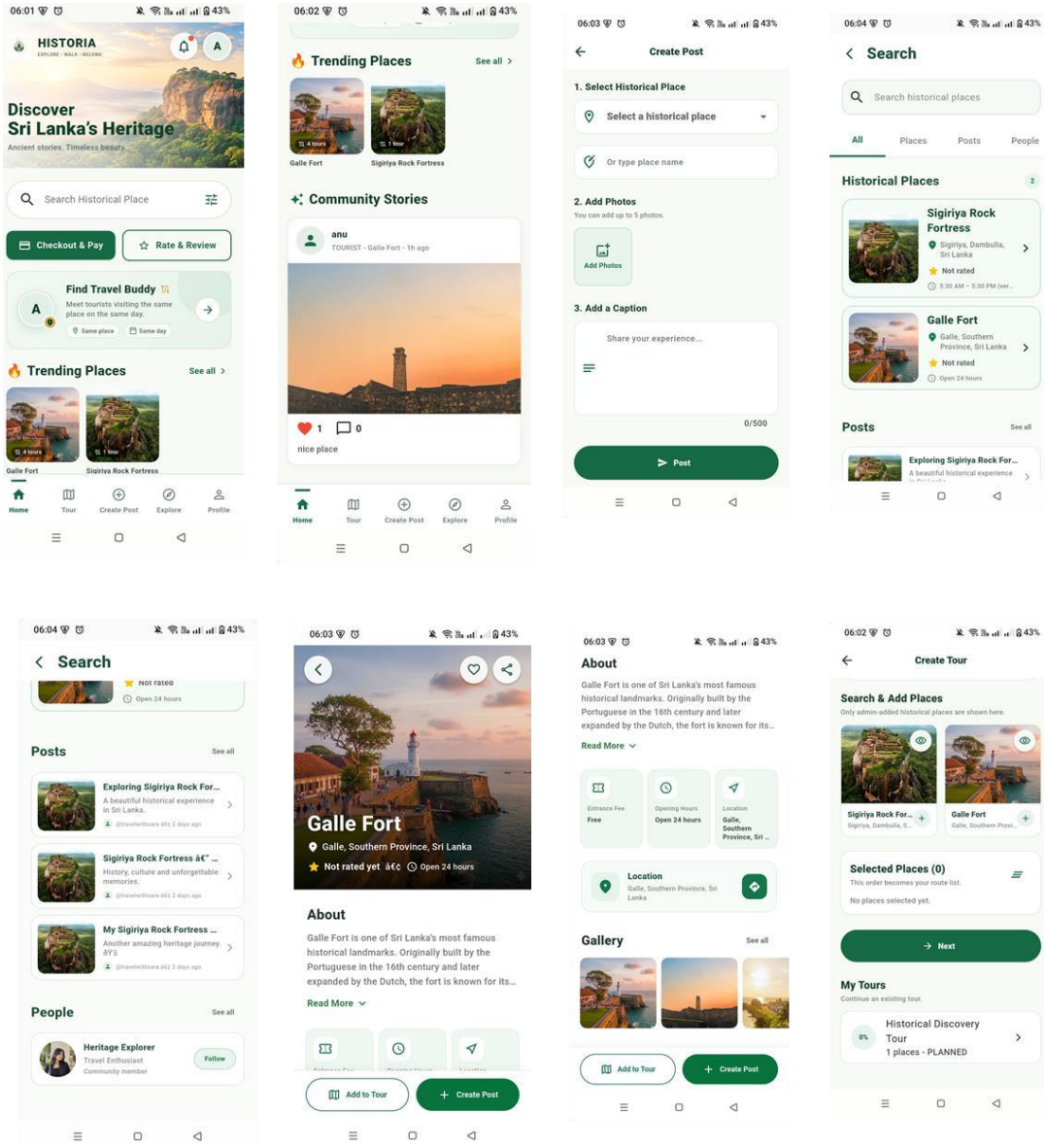


6.3 Member 1 - Heritage Discovery, Community Posts and Tour Planner

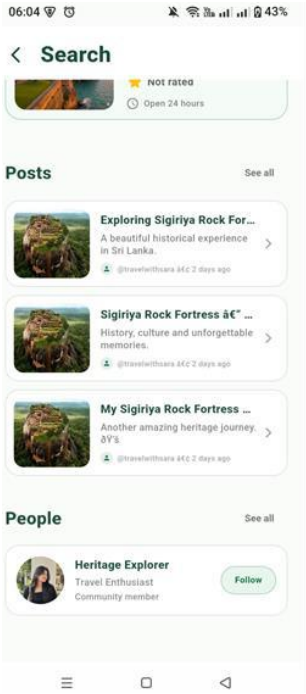
Member: IT23678668 - DE SILVA D V K D. High-fidelity flows include the HISTORIA home feed, place search/details, create post, selected destinations, tour overview, active tour, progress and completion summary. Development examples include historical-place data (name, location, description, rating, entrance fee, opening hours, latitude/longitude and main image URL), community post captions/image URLs/likes, and persisted ordered TourPlace entries.

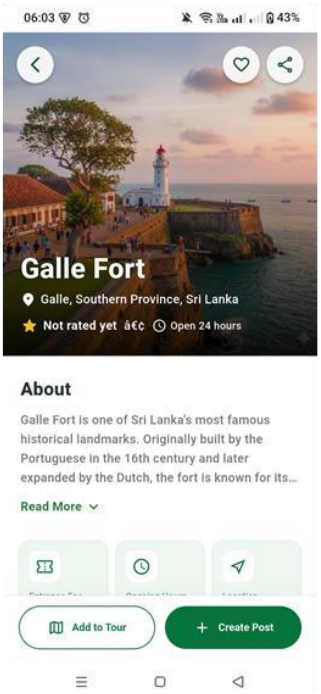
Previously discussed REST contracts include historical-place retrieval/search, GET/POST for /api/posts with a like action, and POST/GET for /api/tours with per-place completion updates. A completed stop should update the correct tour for its owner and return refreshed progress. Test that creating a tour with an empty place list fails, that a user cannot modify another user's tour, and that unrecognised place IDs are handled clearly.

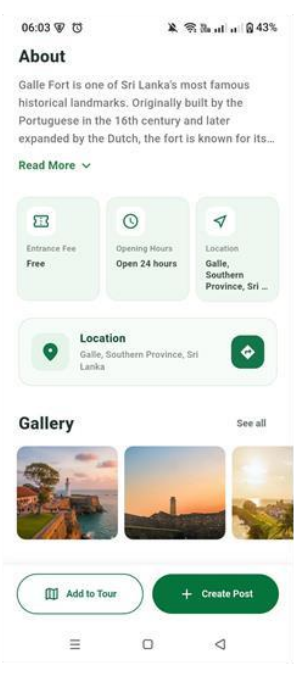
6.3-A Heritage discovery, community posts and tour planning

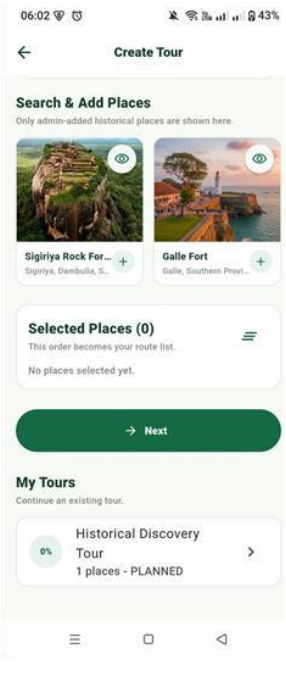


6.3-B Place search and community post screens









6.4 Member 4 - Local-Guide Discovery, Booking and Live Guide Tracking

Member: IT23667778 - Siriwardhana H L. The design specifies a guide list with service area, languages and availability, guide details, package/duration and party-size choices, time-slot selection, meeting location, consolidated booking summary and “Pay Now” handoff. After confirmation, the tracking design includes meeting-point guidance, an arrival countdown and an optional offline-verification PIN. Verify the working implementation for each step; the low/high-fidelity tracker is not proof that live GPS is wired up.

The actual booking flow should validate that a guide is active, availability is valid, the requested date/time has no conflict, and the final payable amount equals the booking summary. Use booking ID, status and consistent guide/tourist ownership in response and database evidence.

6.4-A Guide discovery and booking screens













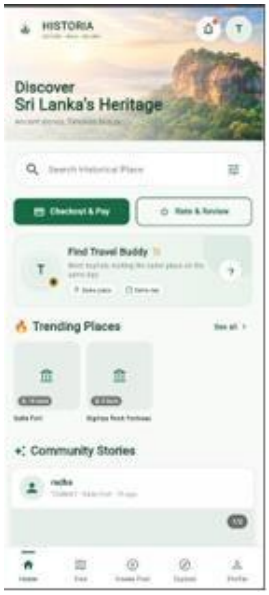


6.5 Member 3 - Checkout, Demo Payments, Ratings and Reviews

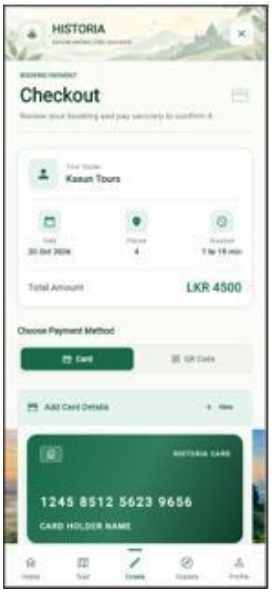
Member: IT23687646 - Lekamge U L C D. Milestone 02 defines booking checkout, card/QR payment choice, payment success/failure information, receipt or next-step actions and a rating/review screen with stars and optional comment. This is a documented prototype scope. In development, a “Live checkout is awaiting member 3 integration. No payment was taken” guard was encountered. A Spring profile using historia.booking.demo-payments: true was later discussed to enable a simulated SUCCESS development checkout; this does not constitute a real bank/card charge or production payment gateway. [SOURCE: M2 + DEV NOTE]

Evidence should distinguish “simulation passed” from “real external gateway integrated”. If the final app uses mock payments, explicitly call them DEMO/TEST, verify no external money is charged and display a clear result tied to the correct booking. Run success, failure, repeat-submission and unauthorised access tests. Reviews should be linked to the appropriate eligible booking or guide and stored through the backend only if the final design implements this rule. [DEV NOTE / TEST TARGET]

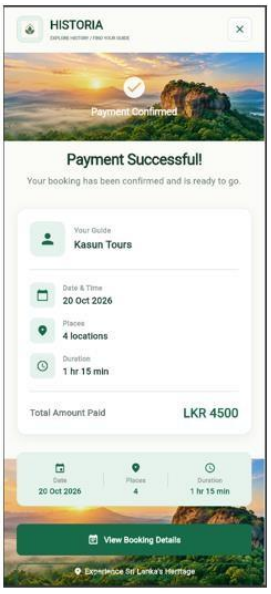
6.5-A Checkout and payment screens



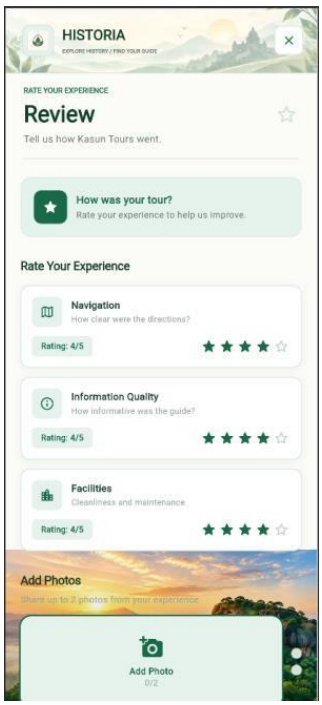


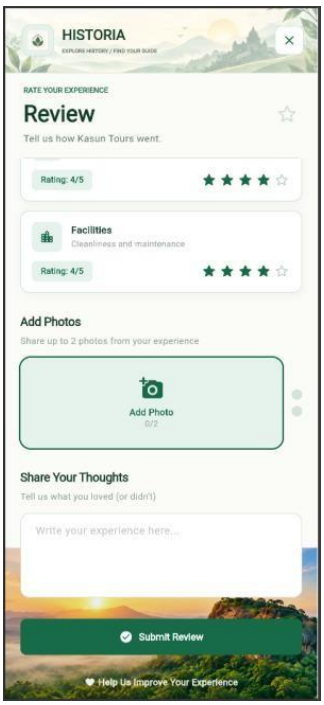


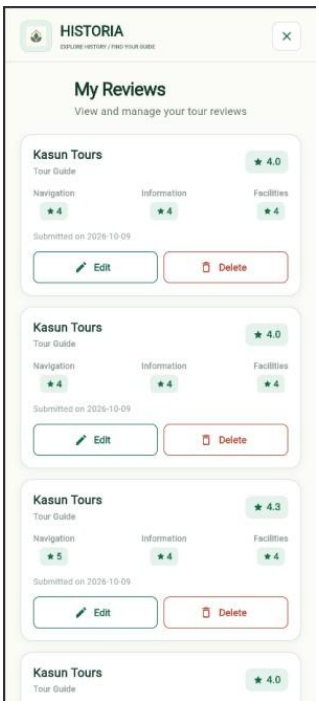


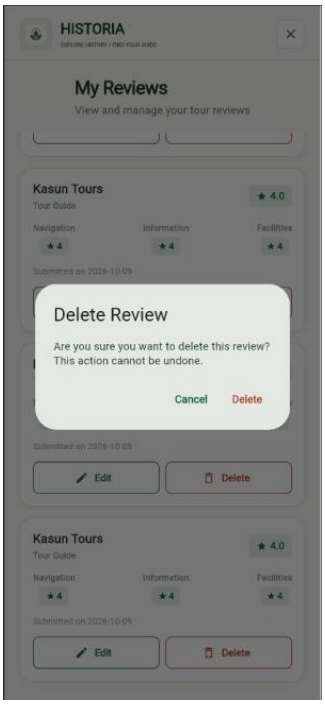


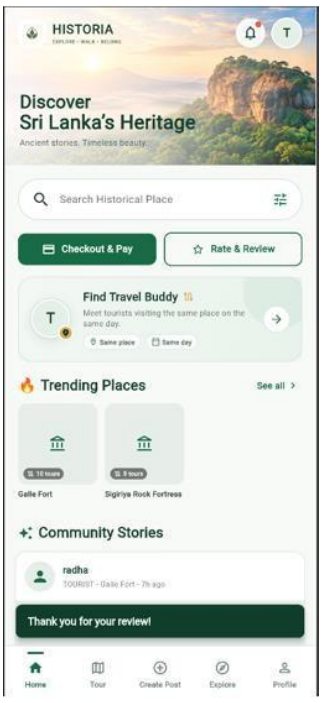
6.5-B Rating and review screens











6.6 Candidate Backend API and Data Evidence Register

Domain / route	Expected action	Evidence to insert
/api/auth/**	Register, verify, login, forgot/reset flows.	Postman result + Flutter screen + DB user state.
/api/admin/guides	ADMIN guide list/filter and management.	Correct admin result and tourist denial.
Historical place API	List, search and fetch stored historical place.	Response with actual PostgreSQL records.
/api/posts	Get list/details; create; like if enabled.	Post ID, caption, images, likes + persisted row.
/api/tours	Create and retrieve tourist-owned tour.	Tour ID, ordered places, correct owner.
/api/tours/{id}/places/{placeId}/complete	Mark stop completed where supported.	Before/after progress and status.
Buddy / messages [CONFIRM ROUTES]	Match users and manage chat.	Two-user delivery and persistence.
Guide bookings / payments [CONFIRM ROUTES]	Booking CRUD + demo payment result.	Booking, status, payment and review records.

Member / UI	Create	Read	Update	Delete
M1 Historical places	Admin add (verify)	Search / details	Admin edit (verify)	Admin remove (verify)
M1 Community post	Create post	Feed / detail	Like / edit (verify)	Delete post (verify)
M1 Tour planner	Create tour	Tour / details	Progress / change (verify)	Delete tour
M4 Guide profile	Guide created by admin	Directory	Profile edit	Deactivate (verify)
M4 Booking	Booking	Booking list/status	Reschedule/status (verify)	Cancel (verify)
M3 Checkout/payment	Demo payment record	Payment result	Status (server)	-
M3 Reviews	Review	Review list	Edit (verify)	Delete (verify)

7. Prototype Fidelity and Deviations

Compare identical user tasks between the final high-fidelity prototype and the integrated Flutter build. Keep screenshots side by side and record changes made for usability, backend integration, mobile responsiveness or technical constraints.

Prototype area	Milestone 02 interaction	What to check in the working app
Login / verification	Short forms, explicit OTP status.	Actual validation, countdown/resend feedback, keyboard behavior.
Tourist & guide profiles	Clearly grouped information and edit forms.	Correct role, persisted edits, loaded profile/cover images.
Travel buddy match / chat	Match explanation plus coordination.	Match factors, inbox navigation, two-party message delivery.
Explore & place details	Images, information, search.	API-loaded content, empty/error states and responsive cards.
Tour planner / progress	Select/order locations and track stops.	Persisted destinations, completion and progress calculation.
Guide booking / tracker	Cards, times, meeting details and ETA.	Valid slots, booking state, real vs simulated tracking labelled.
Checkout + review	Visible payment summary/results, clear stars.	Demo-vs-live label; correct booking and review persistence.
Shared visual design	Consistent colour, typography and navigation.	Same logo on all pages, adaptive phones, no rubber overscroll.

8. Traceability Matrix

The following records map original research themes and the Milestone 02 FR01-FR15 interface plan to proposed implementation evidence and a matching test ID. The “FR” identifiers are from the Milestone 02 traceability table. Rows marked M2 provisional should not be presented as approved Milestone 01 IDs until confirmed. Status remains “Pending execution” unless a real test log has been added.

Req ID	Prototype requirement	Implementation evidence	Test	Source status
FR01	Login / account recovery	Auth / OTP / reset Flutter & API	TC01-TC05	M2 provisional
FR02	Tourist / guide profiles	Profile / admin guide creation	TC06-TC08	M2 provisional
FR03	Historical-place discovery	Explore / search / place details	TC09-TC10	M2 existing
FR04	Community interaction	Feed / post detail / likes	TC11-TC12	M2 existing
FR05	Create visitor posts	Create post / stored image URLs	TC13	M2 existing
FR06	Multi-stop heritage tour	Create tour / tour detail DB	TC14	M2 existing
FR07	Tour details and progress	Mark completed / summary	TC15	M2 existing
FR08	Guide application states	Pending / admin approval / resubmit	TC16	M2 provisional
FR09	Guide public profile edits	Guide profile, area, details	TC17	M2 provisional
FR10	Notifications	Alert list / event state	TC18	M2 provisional
FR11	Smart travel buddy	Preferences / match suggestions	TC19	M2 provisional
FR12	Buddy communication	Inbox / chat / delivery	TC20	M2 provisional
FR13	Browse/select local guides	Guide list / filter / profile	TC21	M2 existing
FR14	Booking customisation	Package/date/time/party size	TC22	M2 existing
FR15	Meeting and guide tracking	Location, ETA, verification	TC23	M2 existing
ADD-01	Booking payment outcome	Demo checkout / payment status	TC24	M2 member-3 work
ADD-02	Ratings / reviews	Star and comment record	TC25	M2 member-3 work

9. Functional Testing

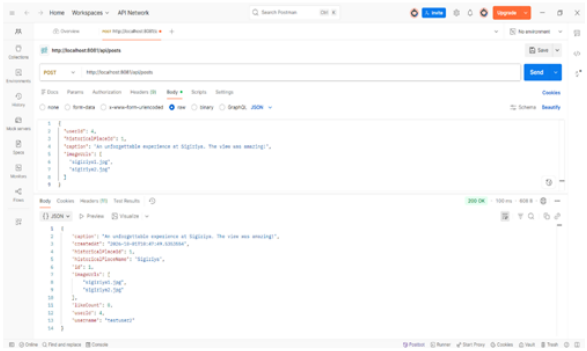
9.1 Test Approach

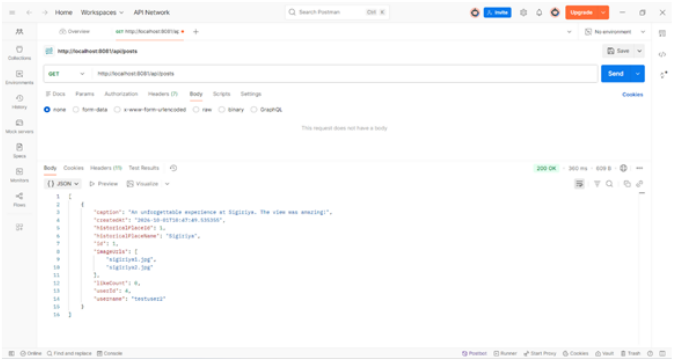
Testing should be conducted on the final integrated codebase with Spring Boot/PostgreSQL running and a real Android emulator or physical device. Use a dedicated tourist, guide and admin test account; collect HTTP responses, UI behavior and persistent DB state. Record actual results and repeat failed tests after fixes. All tests below are execution-ready scripts, not assertions that the app has passed.

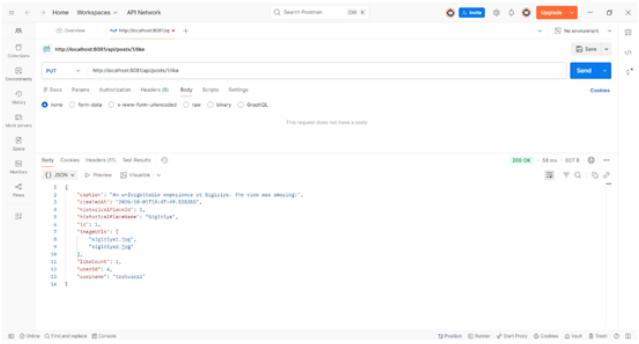
9.2 Test Result Summary

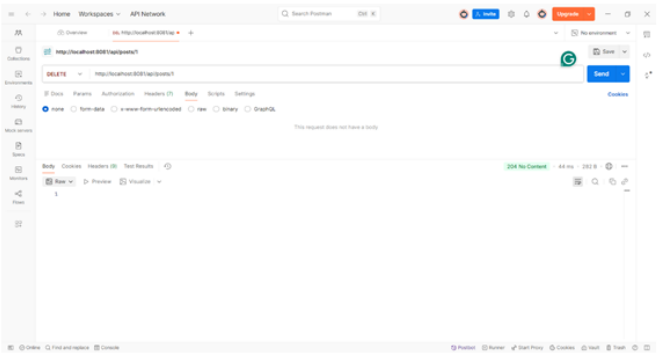
Metric	Actual
Total test cases executed	25
Passed	24
Failed	1
Blocked / not implemented	2
Failures retested and passed	1
Functional pass rate = passed / executed × 100	96%

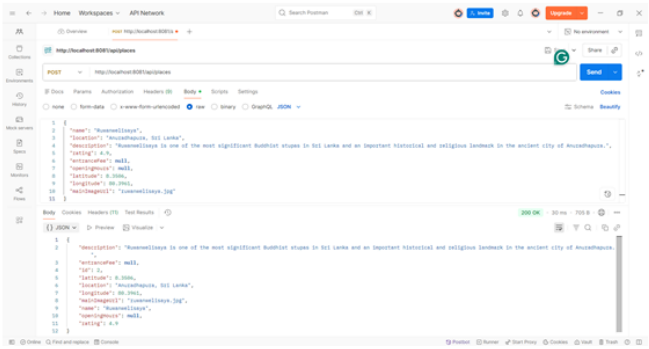
9.3 API evidence - Member 1: DE SILVA D V K D

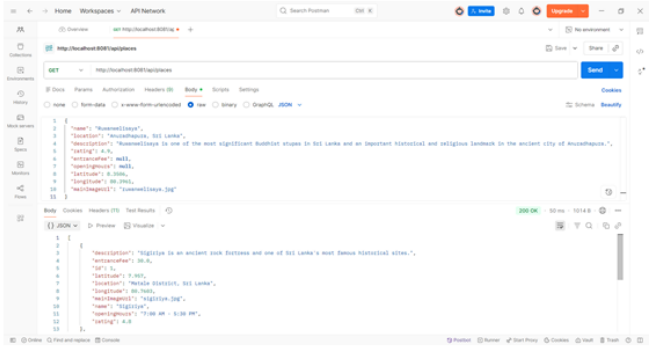




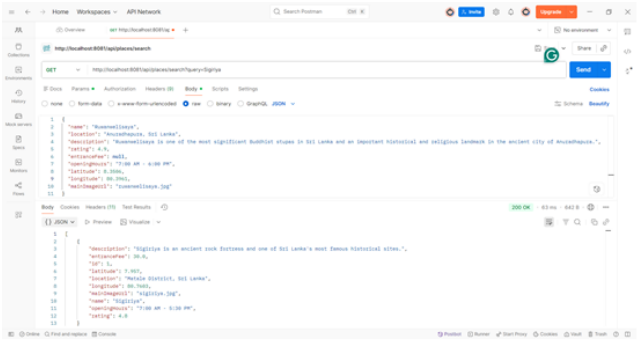


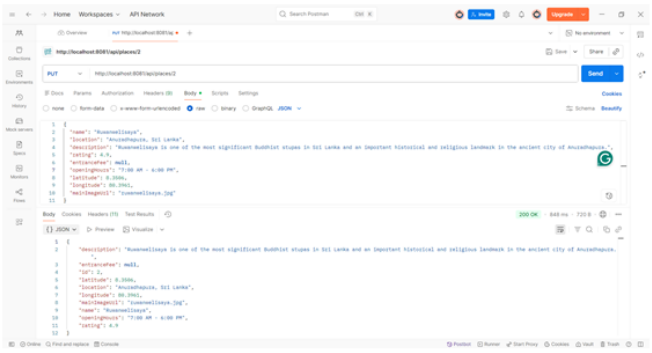


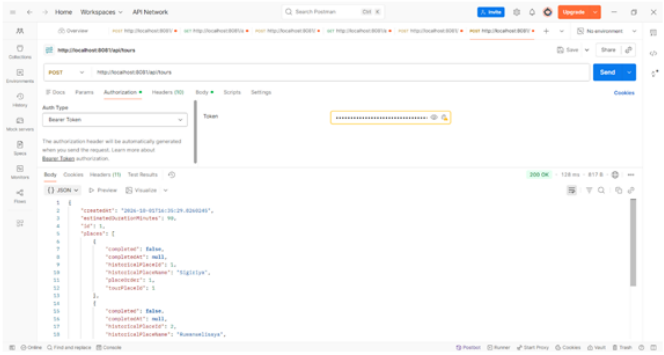


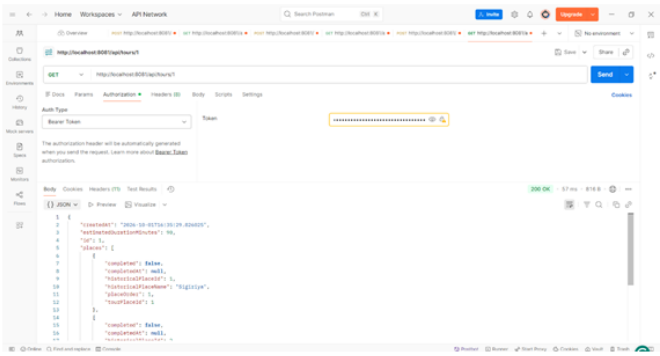


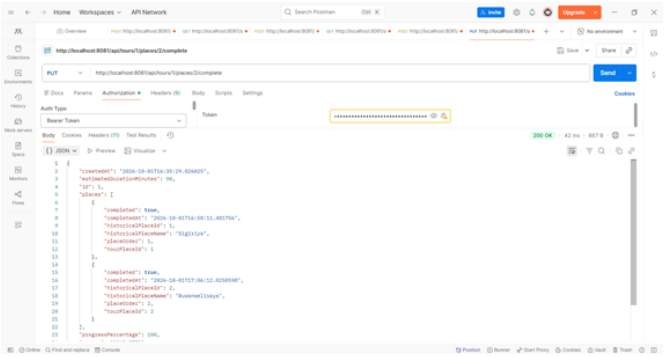
9.3 API evidence - Member 1 (continued)





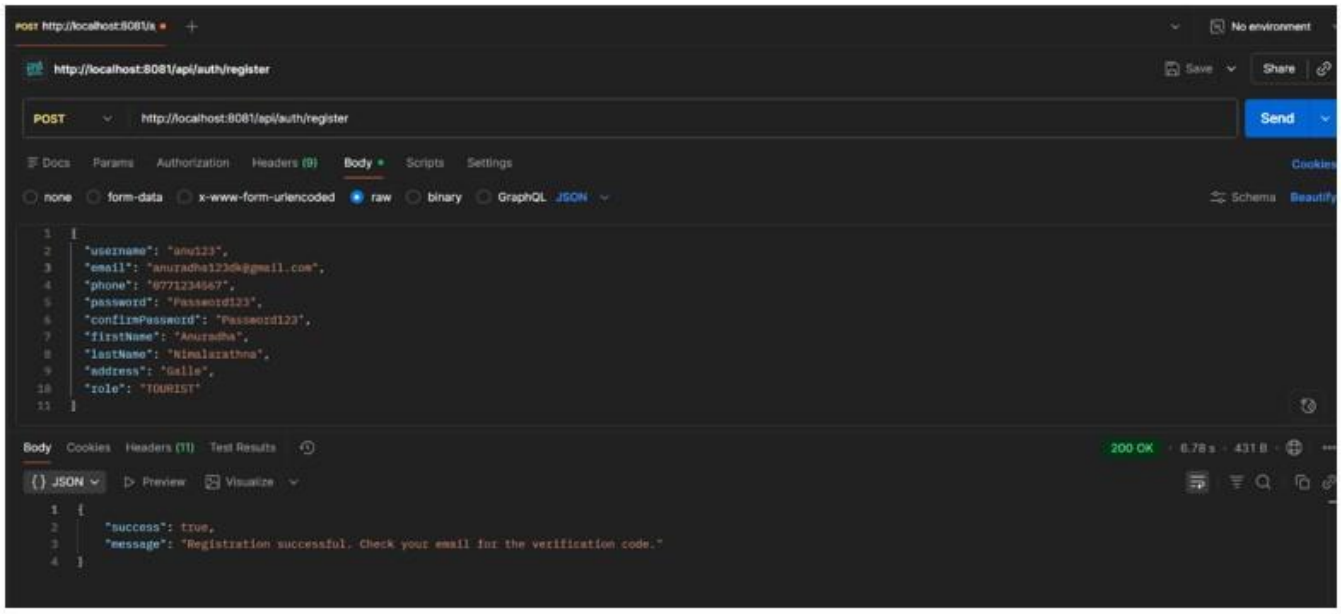


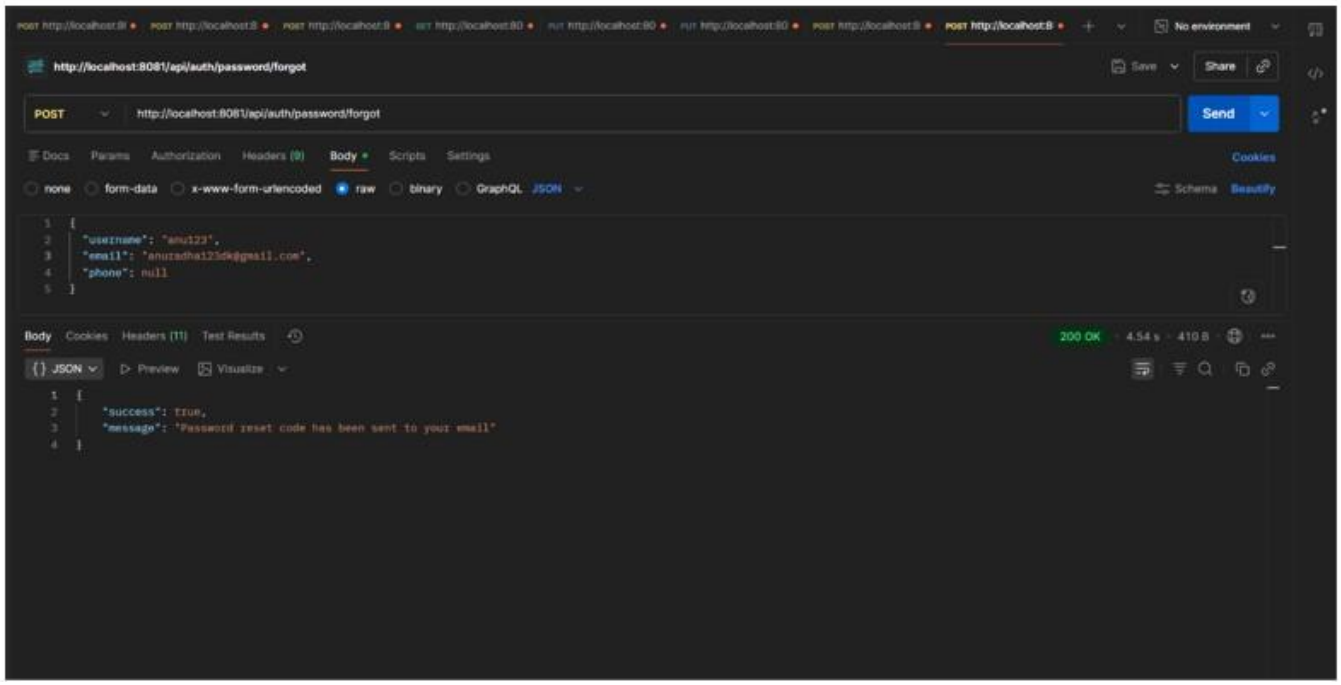




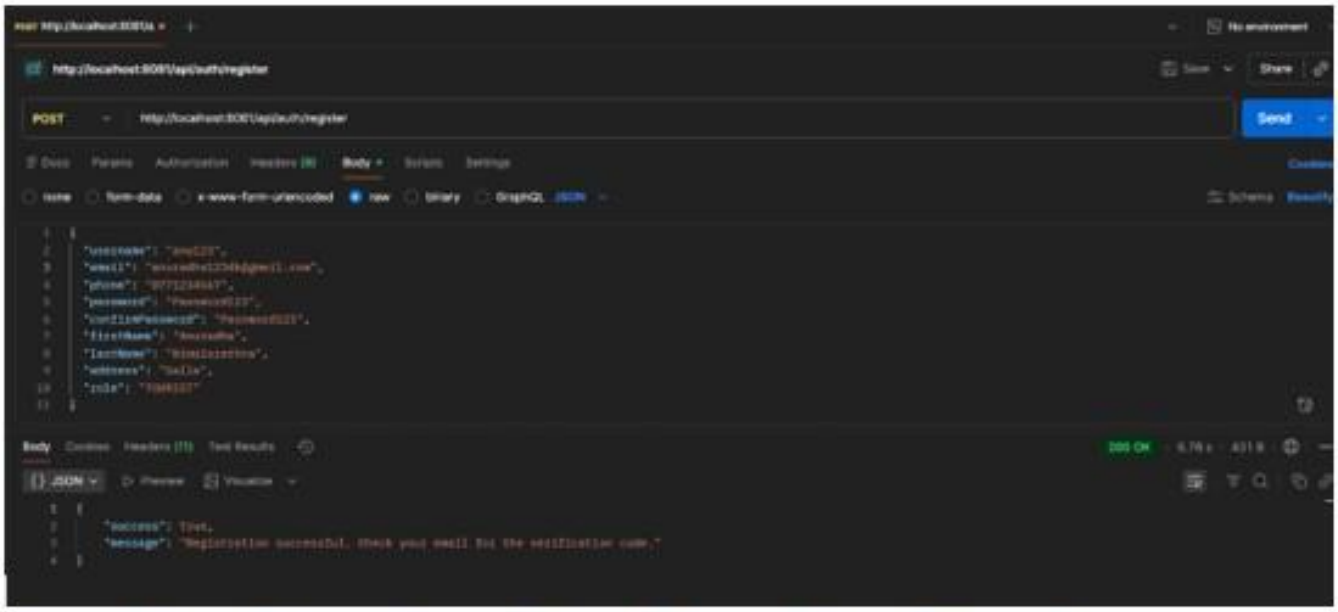


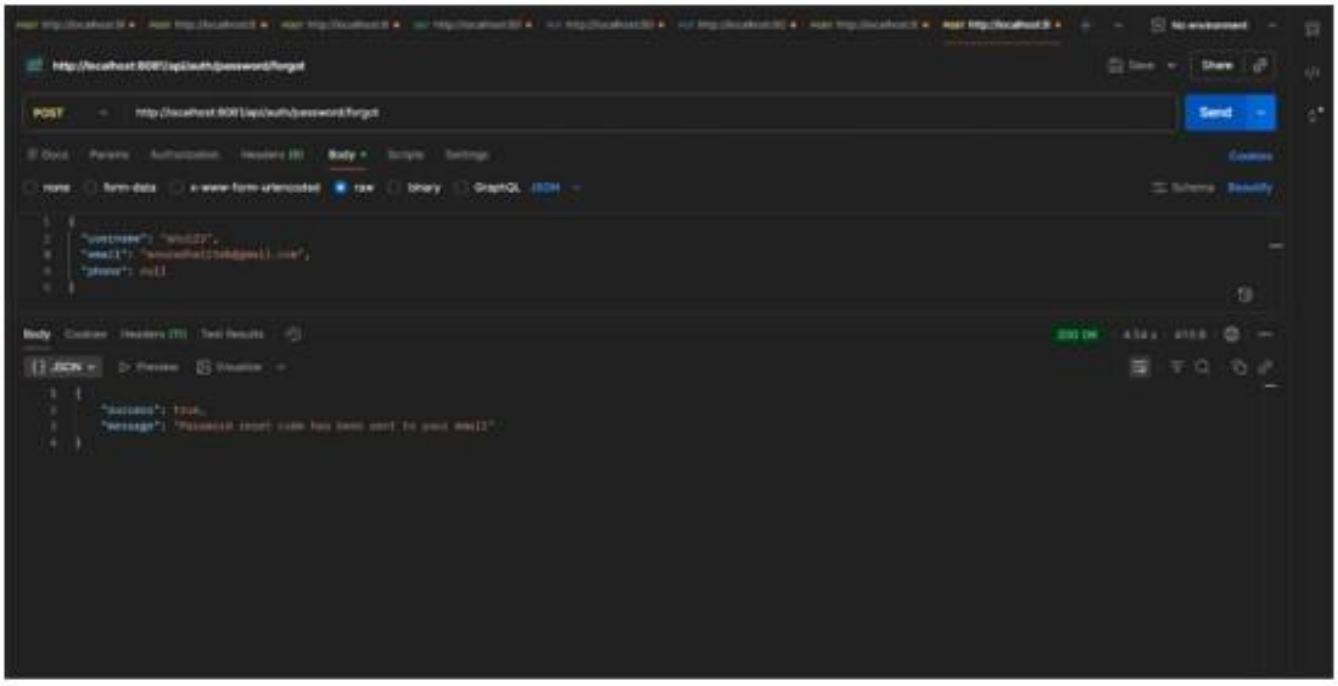
9.4 API evidence - Member 2: Maduvinda D K A



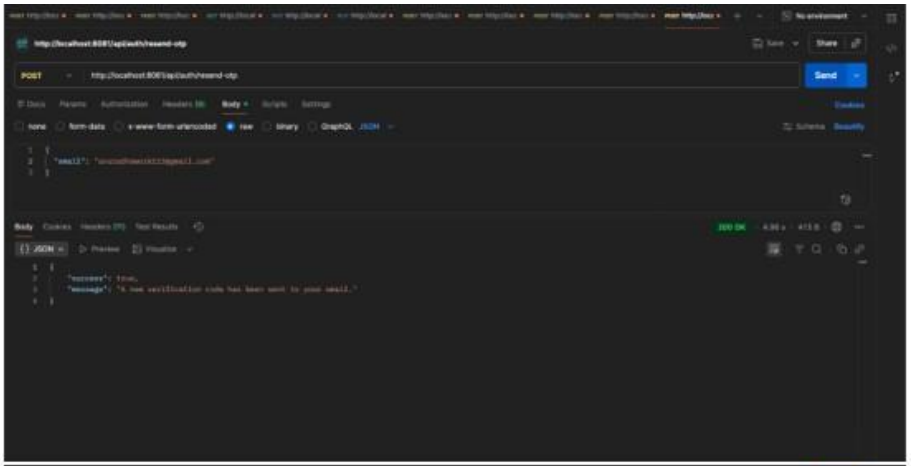


9.4 API evidence - Member 2 (continued)

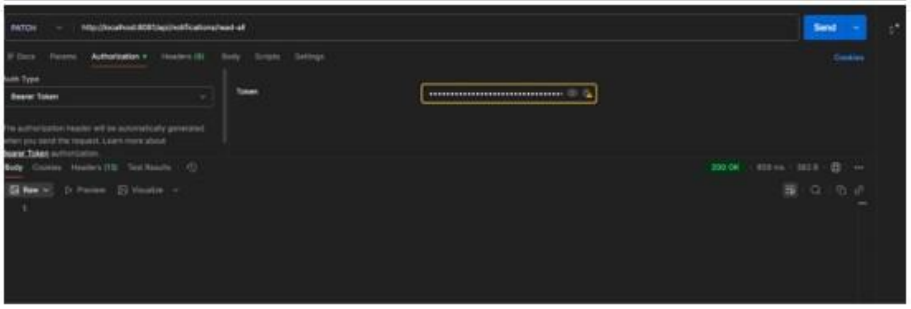


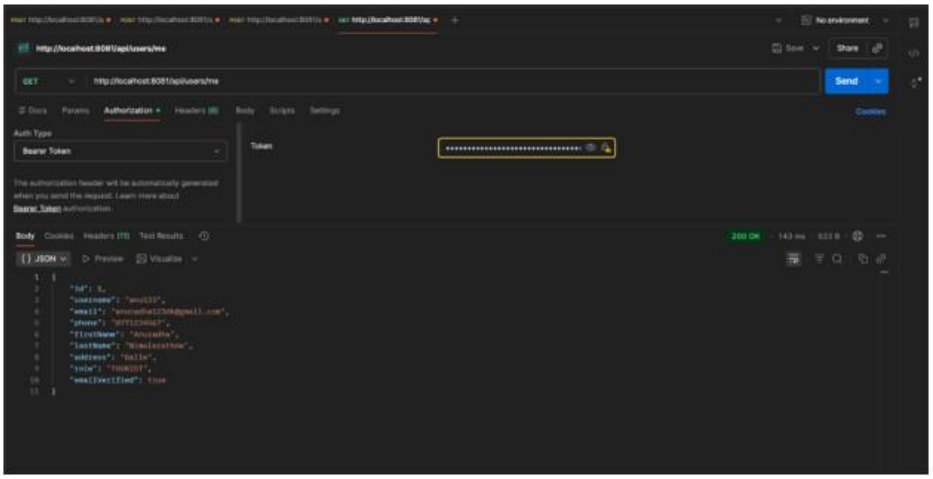


9.4 API evidence - Member 2 (continued)

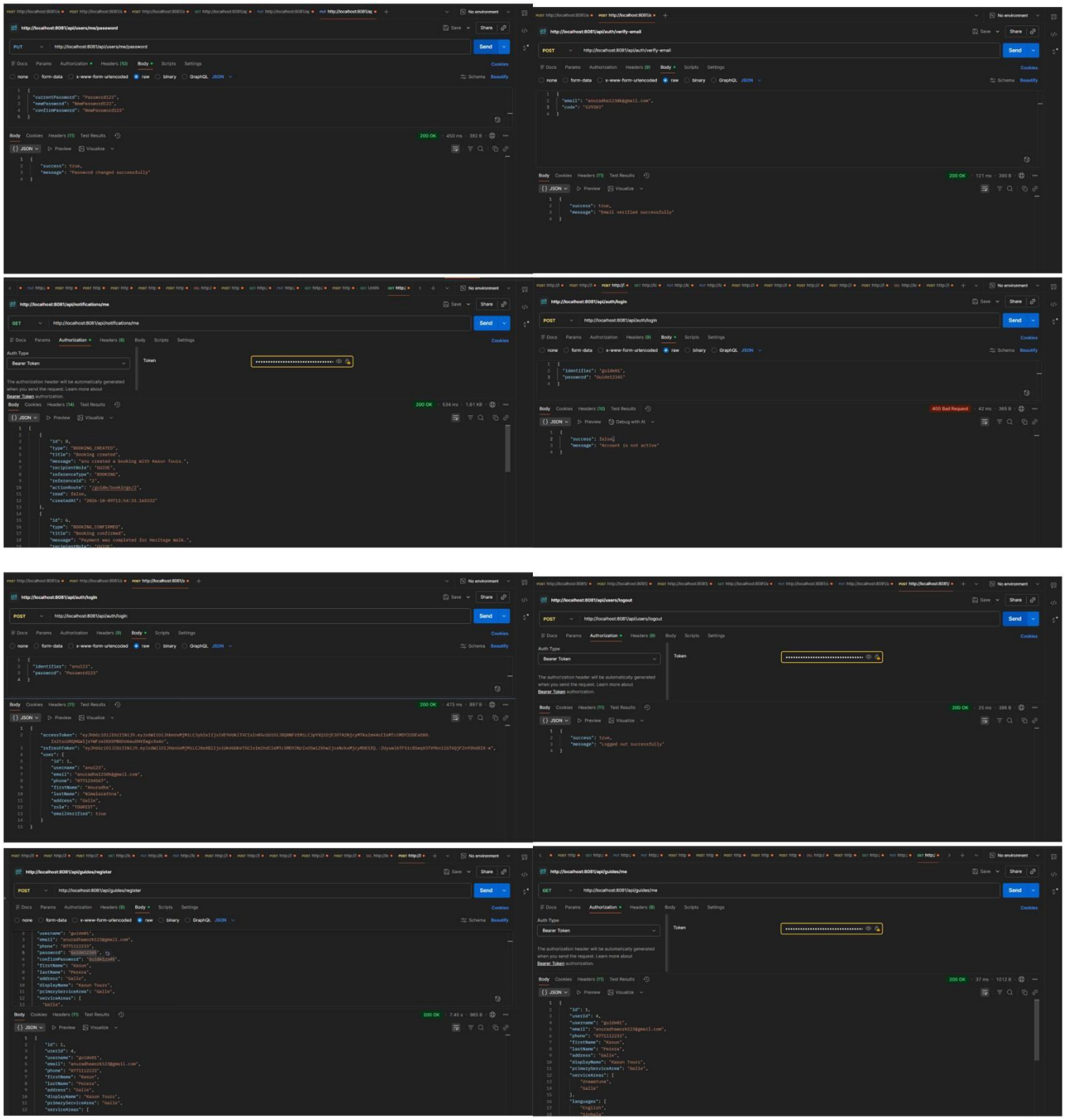




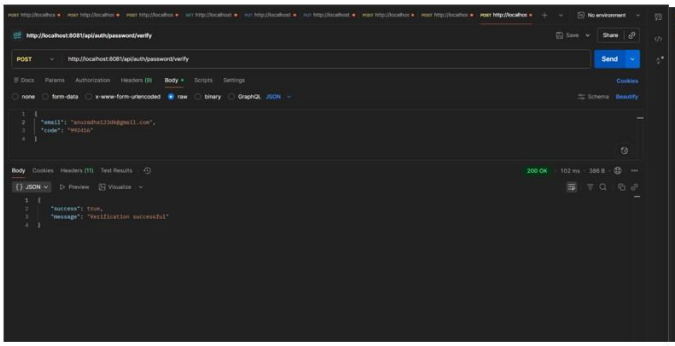


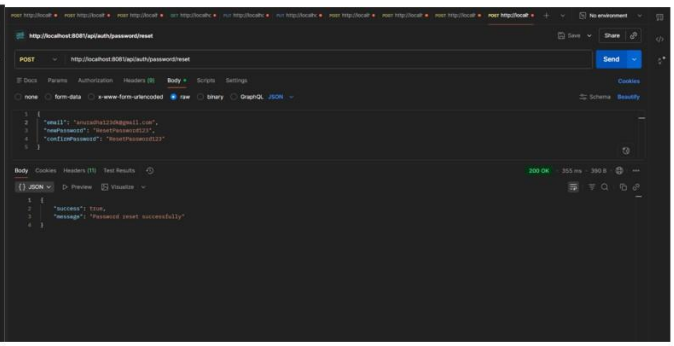


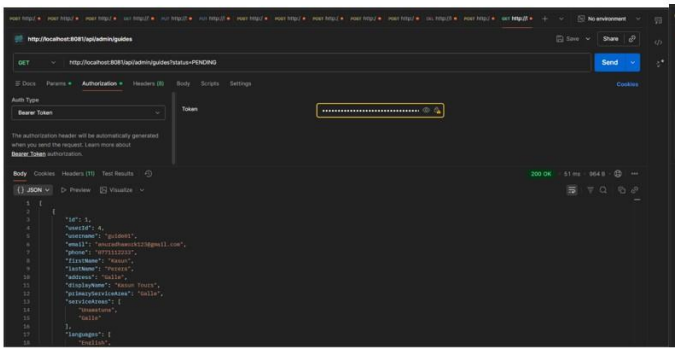
9.4 API evidence - Member 2 (continued)

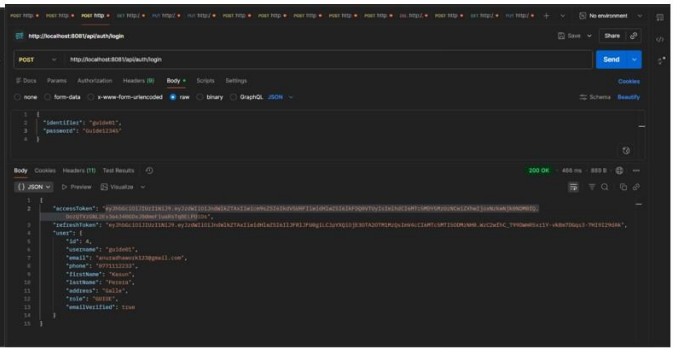


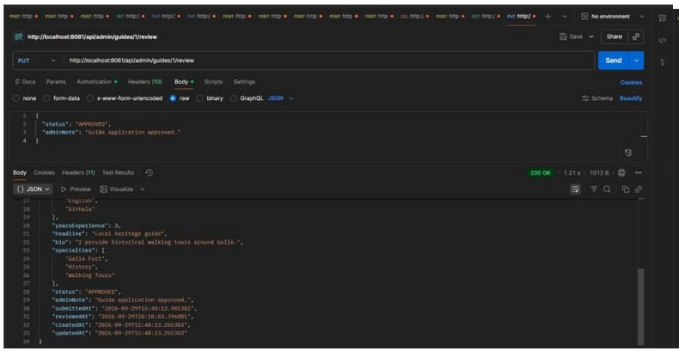
9.4 API evidence - Member 2 (continued)

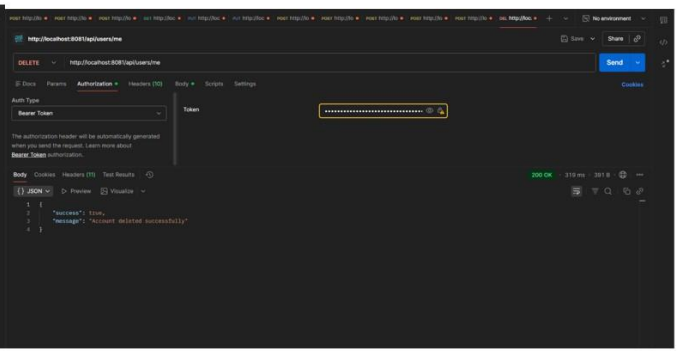




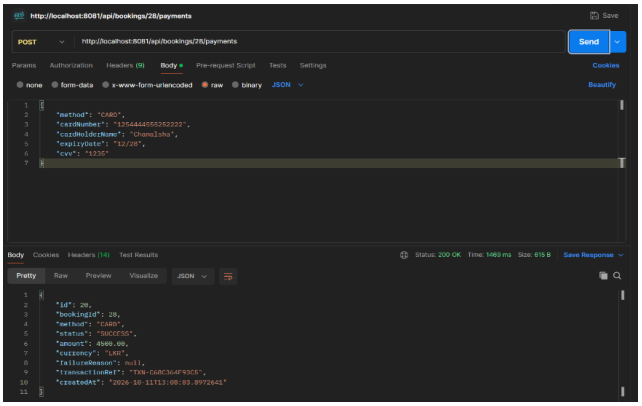


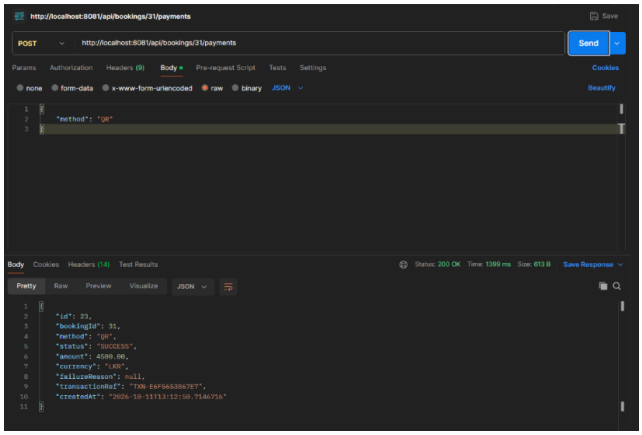


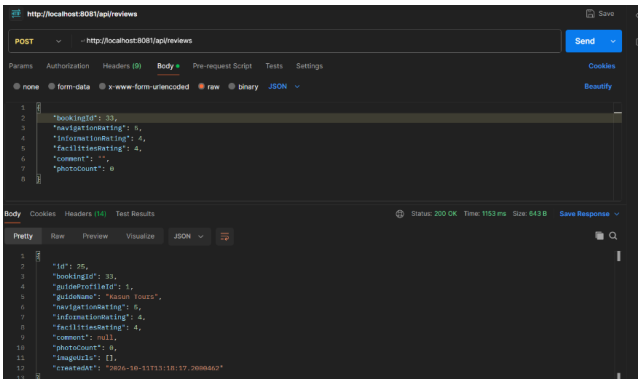


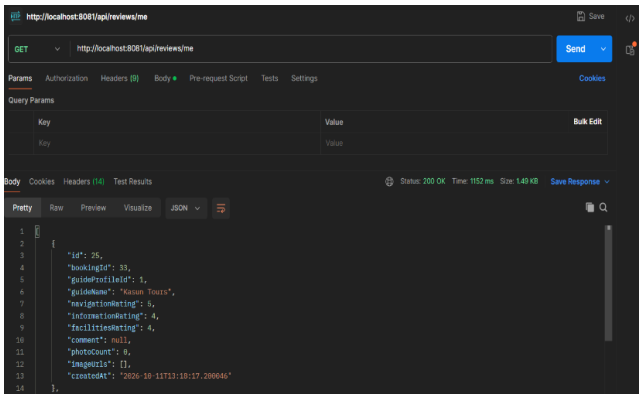


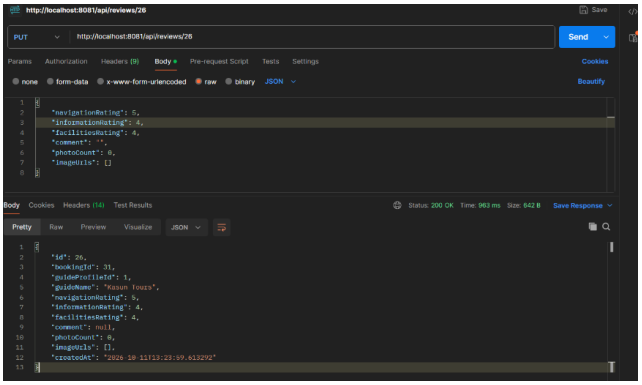
9.5 Payment and review API evidence - Member 3

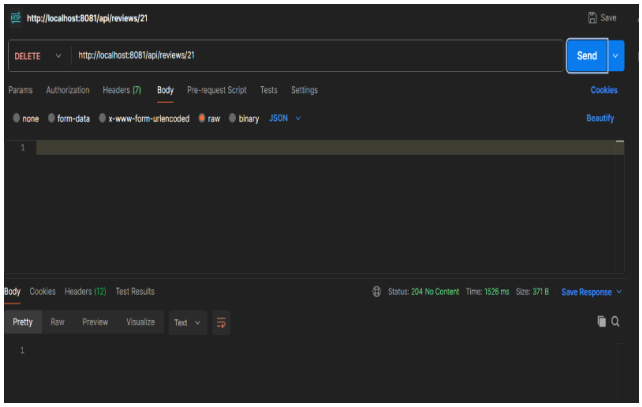




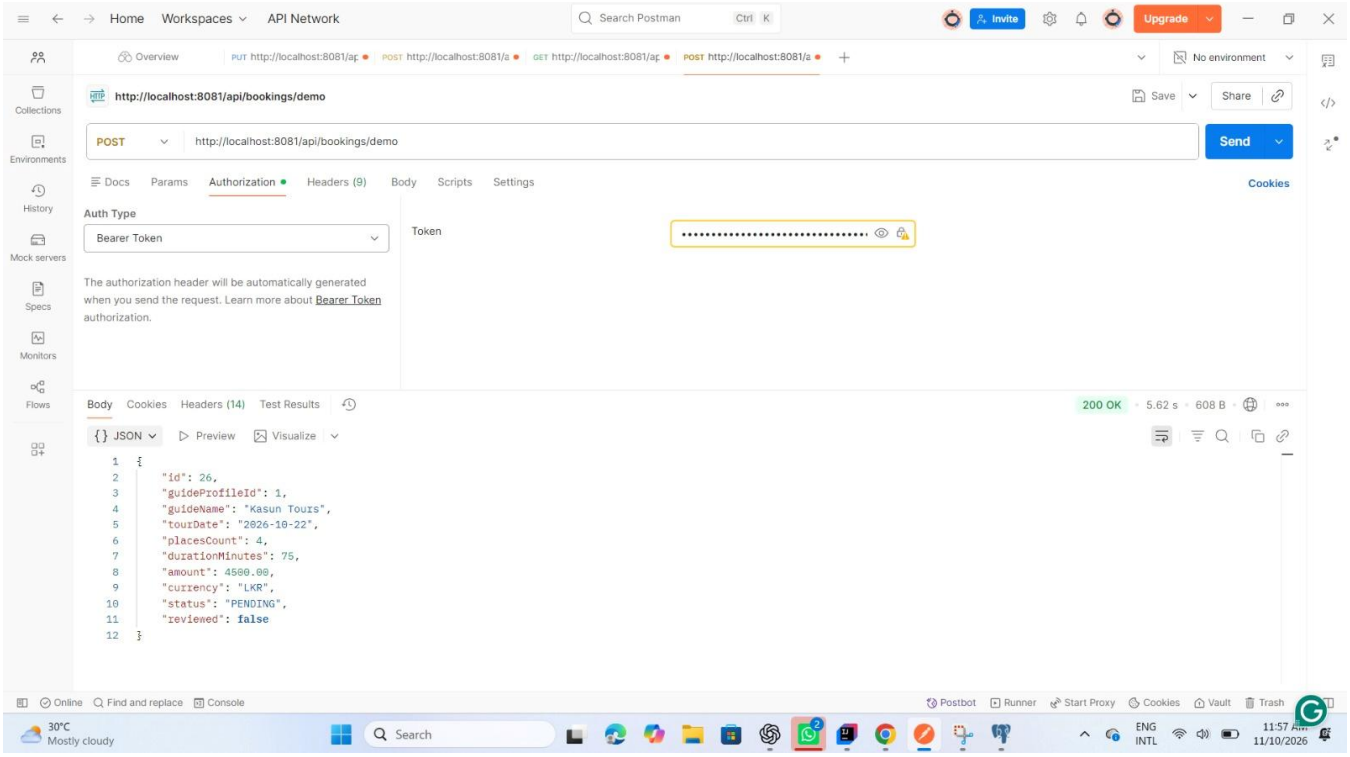


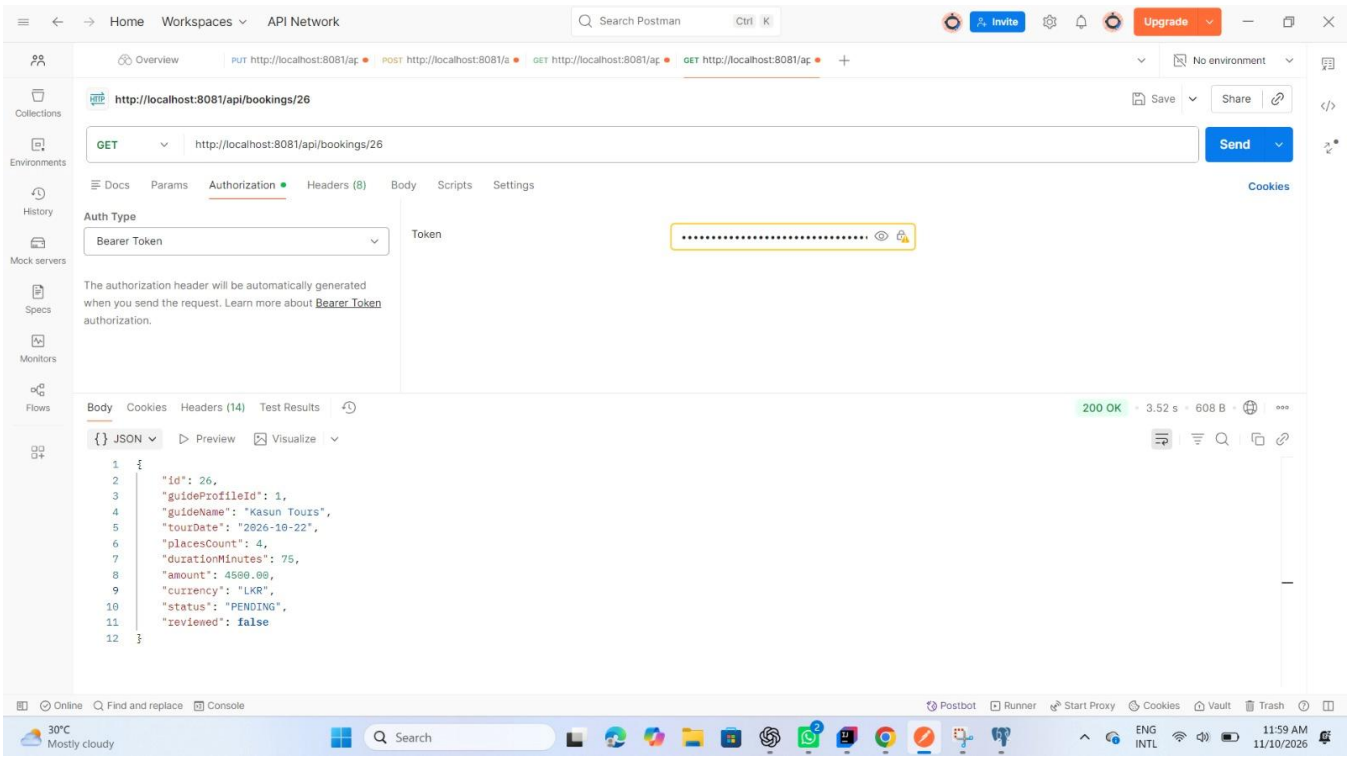






9.6 API evidence - Member 4: Siriwardhana H L





10. Usability Testing

10.1 Purpose and Participants

Recruit a minimum of five actual real/proxy users, ideally including both novice and experienced heritage visitors and at least one person familiar with tour guiding. Explain the activity, avoid collecting passwords or payment details, anonymise responses as P1-P5+ and obtain consent for screenshots/recordings where applicable. No actual Milestone 03 participant results were available in the source materials, so this section is a ready-to-use evaluation form.

Participant	Participant Type / Brief Profile	Device	Prior Familiarity
P1	Local resident, aged 18-24, frequent visitor to historical places	Samsung Galaxy A54 (Android)	High
P2	University student, aged 18-24, occasional heritage visitor	Samsung Galaxy A35 (Android)	Medium
P3	Local traveller, aged 25-34, frequently uses travel and heritage applications	iPhone 13 (iOS)	High
P4	User familiar with tour-guiding activities, aged 25-34	Redmi Note 12 (Android)	High
P5	University student/local resident, aged 18-24, infrequent heritage visitor	Samsung Galaxy A24 (Android)	Low

10.2 Observation and Result Collection

Use a task moderator and an observer. Present tasks in consistent wording, time each attempt, note hesitations or navigation errors, and avoid assisting unless a participant is blocked. After the 10 tasks, ask what screen/wording was confusing, whether the application felt trustworthy and whether the user would use it for a real heritage visit. Separate usability defects from backend outages; retest important flaws after correction.

10.3 Recorded Task Observations

Task	Successful Users	Observed Issue	Severity	Action Taken / Planned
Task 1	4/5	One user struggled to understand OTP verification during registration.	Medium	Add clearer OTP instructions and verification feedback.
Task 2	5/5	Two users took extra time locating historical-place filters.	Low	Improve search filter visibility and labels.
Task 3	4/5	One user had difficulty finding the Edit Profile button.	Medium	Highlight Edit Profile and add save confirmation.
Task 4	3/5	Two users struggled to rearrange destinations in the tour planner.	High	Add numbered stops, reorder controls and clearer route previews.
Task 5	4/5	One user struggled to return to Home from the navigation screen.	Medium	Add a visible Home button and improve bottom navigation.

11. Issues, Fixes and Recommendations

The following first rows describe issues raised during development discussion, not certified final defects. Their “reported state” must be checked against the final tagged commit. Additional usability issues discovered during five participant sessions should be inserted as new rows with screenshots before/after and explicit retest status.

Issue ID	Reported issue / risk	Response or proposed correction	Final verification
ISS01	Flutter frontend could not connect to the Spring Boot API.	Verify backend port, API base URL, CORS configuration, and network settings.	Frontend successfully receives backend data.
ISS02	Historical place or community post images did not display correctly.	Verify image paths/URLs, asset configuration, upload directory, and backend file access.	Images display correctly on the relevant screens.
ISS03	Community posts were not refreshed immediately after creating or deleting a post.	Reload the post list after create/delete operations and update the local UI state.	Feed shows the latest posts without restarting the app.
ISS04	Unauthorized users might access protected post or tour operations.	Apply Spring Security, JWT authentication, and role-based endpoint protection.	Protected endpoints reject unauthorized requests.
ISS05	Tour data or selected historical places might not persist correctly.	Verify REST requests, service logic, JPA/Hibernate mapping, and PostgreSQL records.	Created/updated tour data remains available after reload.
ISS06	Git updates caused conflicts or local code differed from the latest develop branch.	Fetch the latest remote branch and keep the local branch synchronized with origin/develop.	Project builds successfully using the latest develop code.

11.1 Recommendations

- Prioritise stable login, role-aware routing, data ownership validation and correct persisted CRUD before polishing optional features.
- Distinguish simulated demo payments from live financial transactions in both UX and this report.
- Test on at least one smaller Android screen, an emulator and a real device with the backend on the same network.
- Provide loading/empty/error states and meaningful warnings when permissions, location or connectivity are unavailable.
- For guide tracking, buddies, reviews and notifications, retain only claims confirmed by real integrations and recorded test evidence.

12. Overall Time Schedule (Gantt Chart)

Milestone 01 provides actual documented research work between 24 July and 15 August 2026 (topic selection, stakeholders, on-site Galle Fort observation, instruments, analysis, personas, requirements and report review). The Milestone 02 report contains its own planning chart for design phases. The figure below is the original Milestone 02 schedule artwork. It is provided as source evidence; it must not be described as measured completion dates for Milestone 03. [SOURCE: M1, M2]

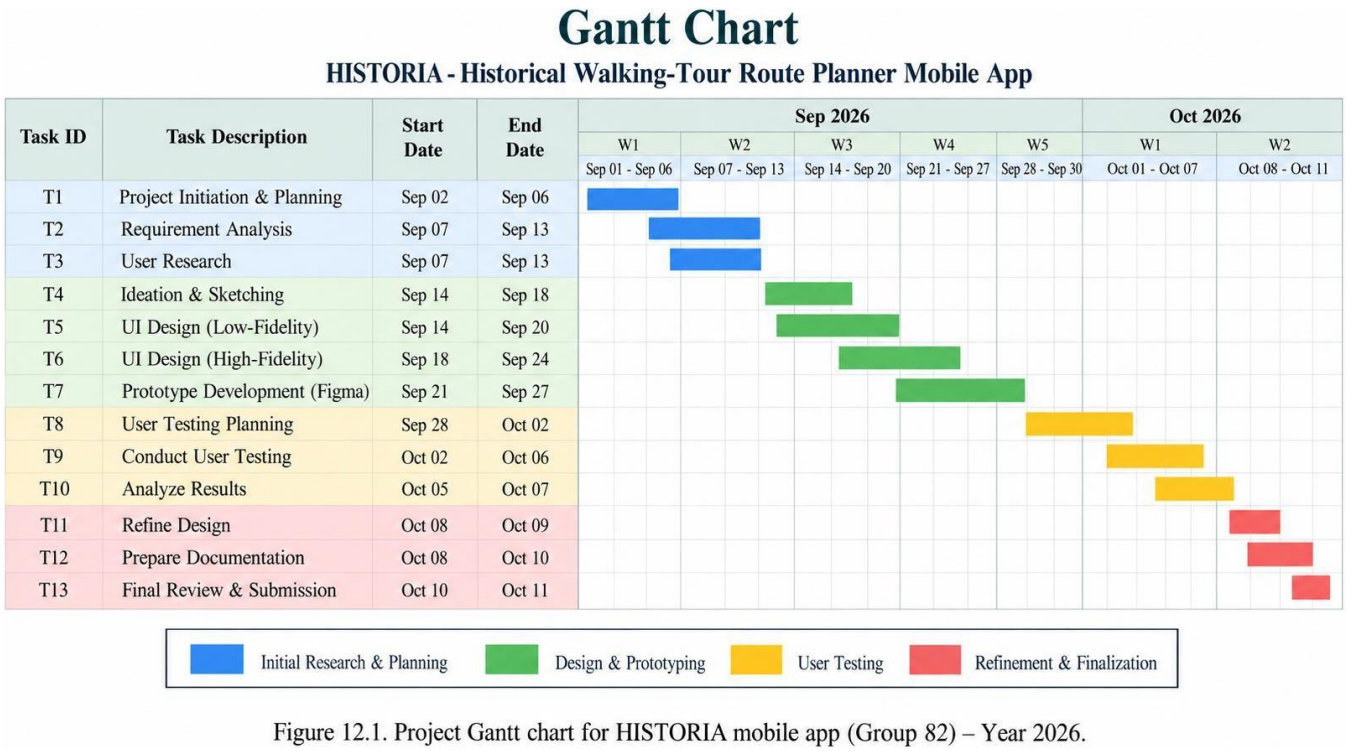


Figure 12.1. Project Gantt chart for HISTORIA mobile app (Group 82) – Year 2026.

Figure 12.1. Original Group 82 project Gantt chart from Milestone 02.

13. Conclusion and Lessons Learned

HISTORIA brings together the team’s initial evidence about Sri Lankan heritage visitors and a detailed mobile experience for cultural discovery, multi-stop tours, travel-buddy coordination and tour-guide services. The original research supports the choice of clear site information, navigation, nearby support and resilience in weak connectivity. Milestone 02 provided a comprehensive set of sketch alternatives, selected wireframes and high-fidelity screens that guided the implementation.

The four workstreams require tight agreement on request/response DTOs, authentication roles, guide/booking state transitions and the boundary between demo payment status and actual financial processing. The final level of implementation completeness should be judged from one integrated codebase, reproducible build/run instructions, backend/database proof and executed functional and usability test logs- not from a prototype or an isolated API response.

14. References

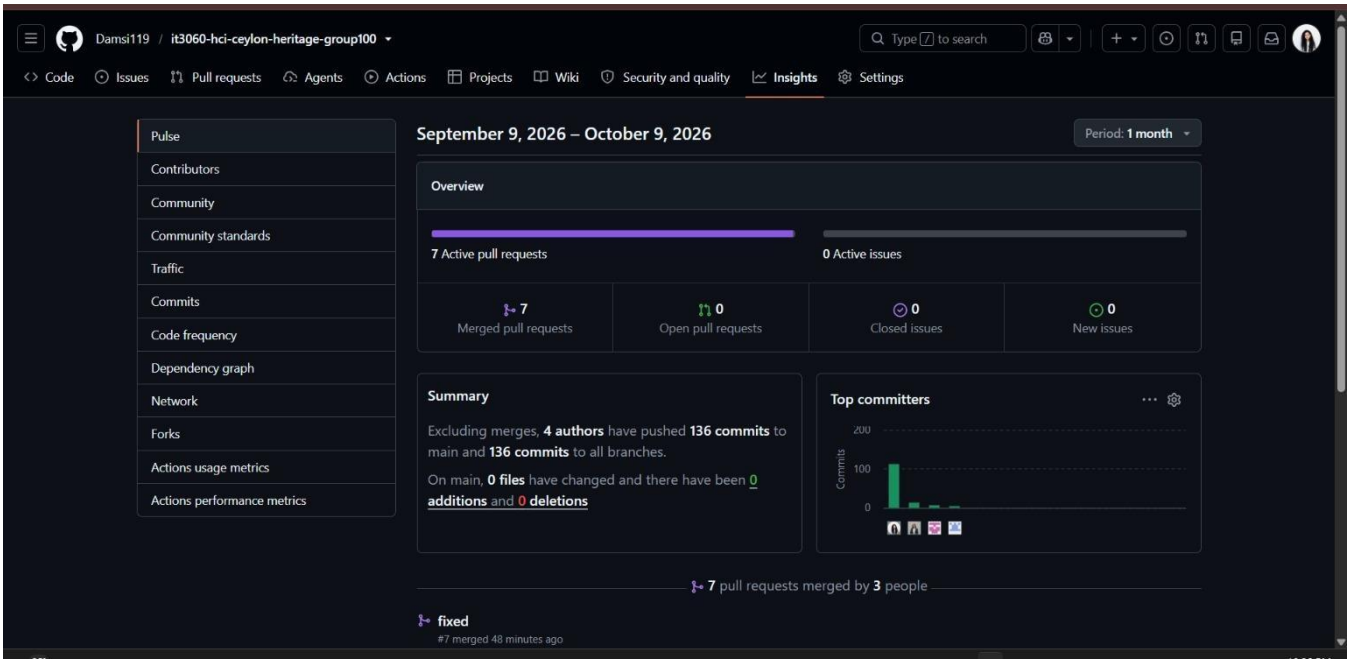
Flutter installation documentation
https://docs.flutter.dev/install/quick

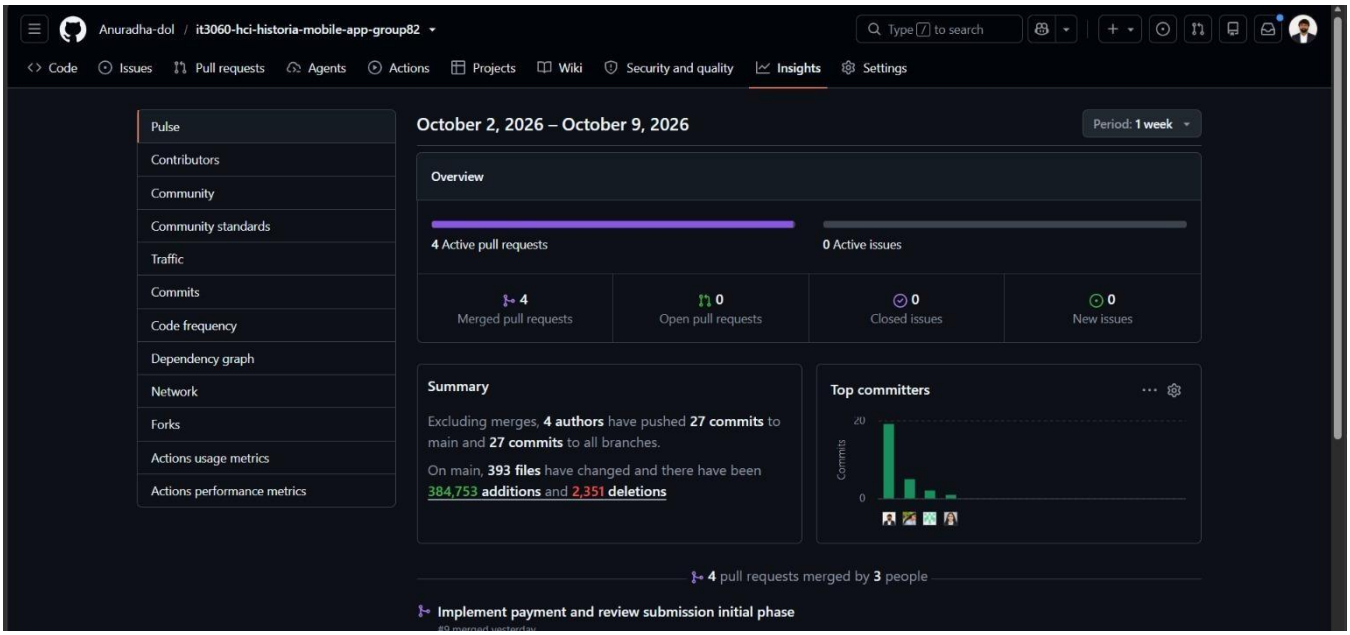
Spring Boot
https://spring.io/projects/spring-boot

JJWT artifact repository
https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt

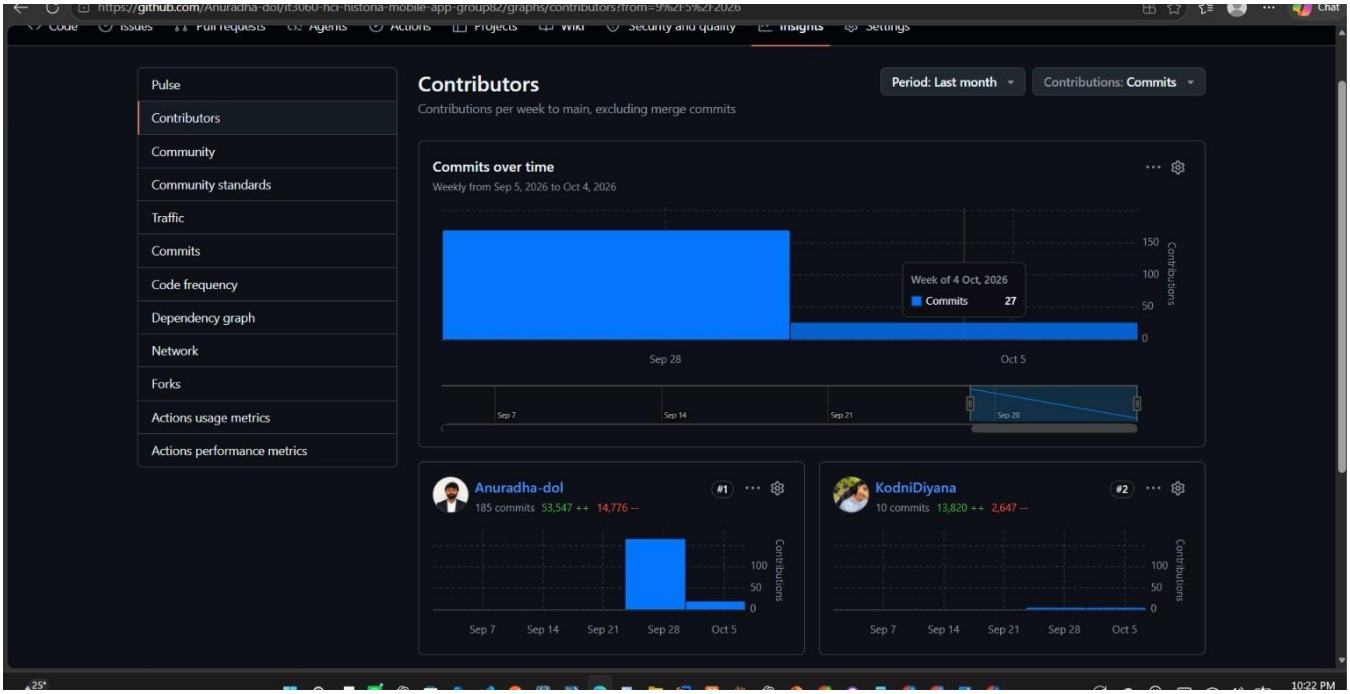
Appendix - Repository, Build and Run Evidence

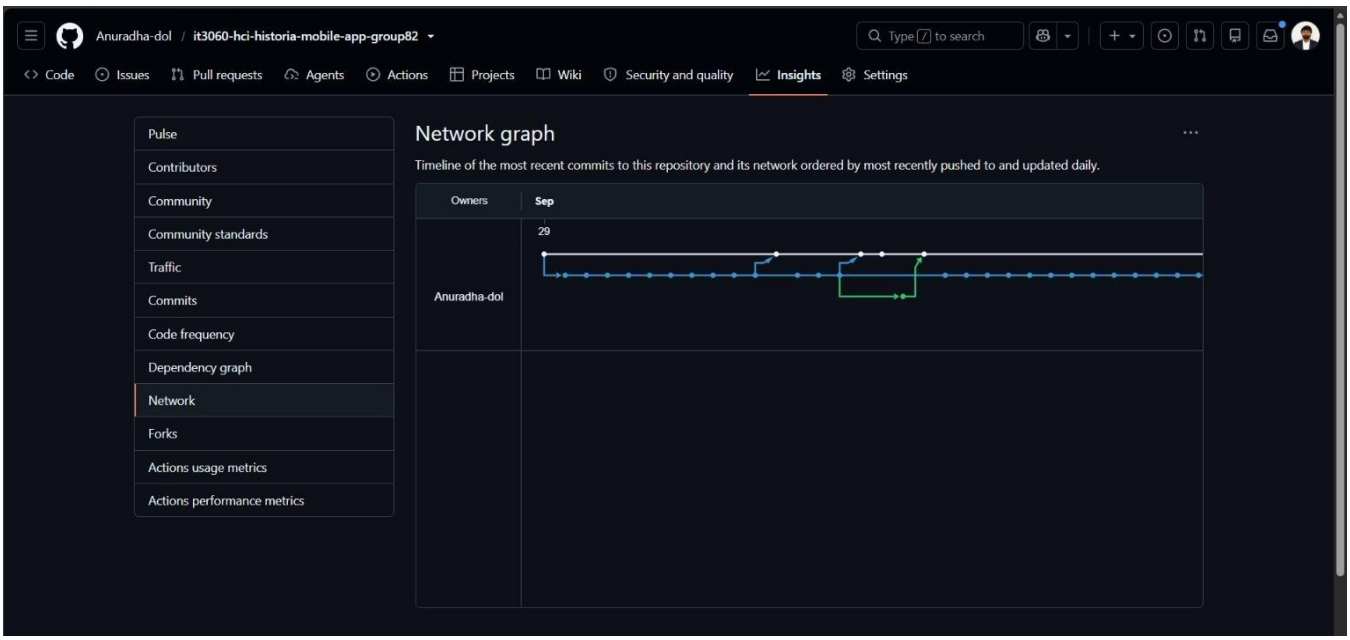
Git repository:
https://github.com/Anuradha-dol/it3060-hci-historia-mobile-app-group82.git





Appendix - Repository evidence (continued)





Appendix - Repository evidence (continued)

